

Bachelorarbeit

Entwicklung eines Geteilten Testbeds für Container- und Netz-Virtualisierung zur Leistungsbewertung von Pub/Sub-Mechanismen

Lars Tönsing

Würzburg, April 27, 2026



Julius-Maximilians-Universität Würzburg
Lehrstuhl für Informatik III
Betreuer: Prof. Dr. Tobias Hoßfeld

Abstract

Die empirische Leistungsbewertung von OPC-UA-Kommunikationsmodellen im großen Maßstab ist auf physischer Hardware aufgrund der Kosten und der logistischen Komplexität großer Knoteninstallationen unpraktikabel. Diese Arbeit entwirft, implementiert und evaluiert ein SR-IOV-basiertes virtuelles Testbed, das über zwei Intel-E810-Netzwerkkarten aufgebaut ist und darauf abzielt, die Zeitverhalten eines physischen Industrienetzwerks nachzubilden, während es bis zu 120 isolierte Anwendungscontainer auf einem einzigen Host betreibt. Das Testbed verwendet SR-IOV im Switchdev-Modus in Kombination mit Software-VLAN-Schnittstellen und hardwareoffloaded TC-Flower-Regeln, um eine vollständige Datenpfadisolierung zwischen Mandanten zu erreichen, ohne den Host-Kernel auf dem Weiterleitungspfad zu durchlaufen. Eine auf der open62541-Bibliothek aufbauende OPC-UA-Anwendung wird in Docker-Containern bereitgestellt und zur Durchführung einer beispielhaften Parameterstudie über Nutzlastgrößen, Publikationsintervalle, Datenpunktverteilungen, Publisher-Anzahlen und Netzwerktopologien verwendet, mit dem Ziel zu beurteilen, ob das Testbed in der Lage ist, aussagekräftige und interpretierbare Ergebnisse zu liefern. Ein einzelnes Vergleichsexperiment mit einer physischen Raspberry-Pi-Umgebung ist enthalten, um zu evaluieren, ob die virtuelle Umgebung die auf realer Hardware beobachteten Trends reproduziert. Die Ergebnisse legen nahe, dass aussagekräftige Messungen gewonnen werden können, wobei die SR-IOV- und Raspberry-Pi-Umgebungen eine richtungskonsistente Latenzskalierung über den bewerteten Bereich hinweg aufweisen. In der untersuchten Viele-zu-eins-Konfiguration erscheint das TCP-Client-Server-Modell ausreichend, wenn nicht sogar vorzuziehen gegenüber UDP Publish-Subscribe, da es eine geringere mittlere Latenz und ein besser vorhersagbares Tail-Verhalten aufweist. Topologieexperimente zeigen, dass die erwartete Latenzreihenfolge über Stern-, Linien- und Baumstrukturen reproduziert wird, obwohl der komprimierte dynamische Bereich einer Single-Host-Softwareimplementierung den Umfang einschränkt, in dem diese Ergebnisse ohne zusätzliche Linkemulation verallgemeinert werden können.

Contents

1	Introduction	4
2	Background	6
2.1	OPC UA	6
2.1.1	Client-Server Communication	6
2.1.2	Publish-Subscribe Communication	8
2.2	Hardware-based I/O Virtualization with PCIe and SR-IOV . .	10
2.2.1	Overview	10
2.2.2	Hardware Support for Device Passthrough	10
2.2.3	Single Root I/O Virtualization (SR-IOV)	12
2.3	Linux Traffic Control	14
2.3.1	Queueing Disciplines (QDisc)	14
2.3.2	TC Flower	15
2.3.3	NAPI	15
2.4	Network Isolation via VLAN	16
2.5	Namespaces	16
2.6	Containerization via Docker	17
2.7	Integration	18
2.7.1	Integration of SRIOV with docker environments	18
2.7.2	Applying Queueing Disciplines in SR-IOV	19
2.7.3	Implementing VLANs in SR-IOV	19
3	Related Work	21
3.1	OPC UA Communication Models	21
3.2	Building Testbeds using Network Virtualization and SR-IOV .	22
3.3	This Work Differs	22
	Acknowledgements	24
4	Methodology	25
4.1	Testbed	25
4.1.1	Hardware	25
4.1.2	Software	26
4.1.3	Network Architecture	26
4.1.4	Custom Topology	32
4.2	Raspberry Pi Testbed	33
4.3	OPC UA Application Implementation	34
4.3.1	Pub/Sub using UDP	34
4.3.2	Client/Server using TCP	35

4.4	Parameter study	35
4.4.1	Scaling Data points	36
4.4.2	Scaling Intervals	37
4.4.3	Distribution Data points	38
4.4.4	Scaling Publishers	39
4.4.5	Different Topologies	40
4.4.6	SR-IOV and Raspberry Pi Comparison	41
4.5	Logging	42
4.5.1	CPU Usage	42
4.5.2	Bandwidth	43
4.5.3	Latency	44
5	Evaluation	47
5.1	Scaling Data Points	47
5.2	Scaling Intervals	51
5.3	Distribution	55
5.4	Publisher Scaling	60
5.5	Different Topologies	65
5.6	SRIOV and PI comparison	68
6	Limitations and Future Work	77
6.1	Scope of OPC UA Communication Models	77
6.2	Uncoordinated Publishing Intervals	77
6.3	Single Subscriber	78
6.4	Limited Physical Validation	78
6.5	Experimental Depth	78
6.6	Quantifying Testbed Fidelity Against Physical Reference . . .	79
7	Conclusion	80

1 Introduction

OPC Unified Architecture (OPC UA) is a platform-independent, service-oriented communication standard for secure and interoperable data exchange in industrial automation systems, standardized in the IEC 62541 series. It has emerged as a foundational technology for the realization of Industry 4.0 architectures, bridging the gap between field-level devices, control systems, and enterprise applications across heterogeneous vendor environments [1]. As industrial networks evolve toward fully connected smart factory paradigms, their scale grows accordingly. It is expected that hundreds to thousands of sensors, actuators, and programmable logic controllers will be interconnected within a single industrial plant [2]. The client-server communication model originally defined by OPC UA can be insufficient to address the one-to-many data distribution requirements of such large-scale deployments [3, Annex B]. To resolve this, the OPC Foundation released Part 14 of the OPC UA specification in 2018, introducing the Publish-Subscribe (PubSub) extension, which enables scalable, broker-based and brokerless data distribution from publishers to an arbitrary number of subscribers without the overhead of individual session management [3].

Given the scale at which such networks must operate, empirical testing of OPC UA PubSub performance in production environments is impractical without substantial prior evaluation. Deploying a physical network topology consisting of hundreds of nodes solely for the purpose of protocol benchmarking represents a considerable capital and operational expenditure, and introduces logistical constraints that are incompatible with iterative development workflows [4, 5]. Software-based emulation and virtualization approaches address this problem by enabling the instantiation of multi-node network topologies on a reduced set of physical machines, thereby significantly reducing both cost and setup time while retaining configurability and reproducibility of experimental conditions [5].

However, software-based network virtualization introduces systematic distortions that affect the validity of performance measurements. Communication between virtualized nodes is mediated by the hypervisor, requiring additional processing steps such as context switching, packet copying, and virtual device emulation. These mechanisms introduce variable latency and jitter that are not present in physically realized networks and can significantly alter observed timing behavior [6]. Importantly, this discrepancy becomes more pronounced as the number of virtualized nodes increases [5]. Since virtual machines or containers share underlying physical resources, contention for CPU time and I/O processing leads to scheduling delays that directly impact network timing. As a result, purely software-emulated testbeds may fail

to accurately reproduce the temporal characteristics of large-scale industrial networks.

Single Root I/O Virtualization (SR-IOV) offers a hardware-assisted solution to this problem. SR-IOV is a PCI Express extension that enables a single physical network adapter to be partitioned into one Physical Function and multiple Virtual Functions, each of which is assigned directly to a virtual machine and communicates with the physical NIC without traversal of the hypervisor software switch layer. This eliminates the buffer copy operations and interrupt virtualization overhead that are the primary sources of latency inflation in conventional virtual networking, while simultaneously providing hardware-enforced isolation between virtual nodes. By routing all inter-node traffic through physical networking hardware, an SR-IOV-based testbed produces latency and jitter characteristics that are representative of real network behavior. This principle is demonstrated by HVNet [7], a framework that constructs virtualized network topologies on a single host using SR-IOV and real networking hardware, achieving latency measurement accuracy up to three orders of magnitude better than purely software-based emulation approaches such as Mininet [8].

The objective of this thesis is to design, implement, and evaluate an SR-IOV-based testbed, and to use it for the systematic performance comparison of the OPC UA PubSub and Client-Server communication paradigms in a multi-node system. The testbed shall faithfully emulate the timing characteristics of a physical industrial network without the cost and inflexibility of a full hardware deployment, enabling repeatable and controlled benchmarking of latency and jitter under varying node counts and network loads. To validate the fidelity of the virtual environment, the results obtained from the SR-IOV testbed are directly compared against measurements taken from an equivalent physical setup built using Raspberry Pi nodes, assessing how accurately the virtualized testbed can reproduce real-world communication behavior across both paradigms.

2 Background

This chapter provides the contextual foundation for the present study. It traces the development of the relevant field, outlines the key concepts and theoretical frameworks that inform this work, and situates the research within the broader academic discourse.

2.1 OPC UA

OPC Unified Architecture (OPC UA) is a platform-independent communication standard for industrial automation, standardized as IEC 62541 [9]. It defines a transport-independent protocol stack, a set of standardized services, and a structured information model for data exchange between devices, control systems, and applications. OPC UA supersedes earlier OPC specifications that were based on Microsoft COM/DCOM, eliminating the dependency on Windows-based infrastructure and enabling deployment on embedded devices, industrial controllers, and general-purpose computing systems alike [9].

The central component of OPC UA is its information model, in which all data is represented as typed nodes within a hierarchical address space rather than being transmitted as raw values. Each node belongs to a defined node class and encapsulates variables, methods, or events alongside their typed relationships, forming a semantically defined graph that is exposed by the server as a structured, queryable representation of system state [10].

The positioning of OPC UA within the ISO/OSI reference model is illustrated in Figure 1. The physical and data link layers are governed by IEEE 802.3 Ethernet [11] and IEEE 802.1 bridging standards [12]. Above the transport layer, OPC UA is transport-independent and defines two distinct communication models, each with its own protocol stack and transport bindings. Client-server communication is implemented over TCP using UA Binary encoding [13], while Publish-Subscribe communication supports both brokerless UDP transport using UADP encoding and broker-based transport over MQTT or AMQP [3]. These two communication models are described in detail in the following sections.

2.1.1 Client-Server Communication

OPC UA defines a client-server communication model based on a request-response pattern [9, 13]. A client establishes a persistent connection to a server and interacts with the server’s address space by invoking standardized services. The address space is a hierarchical, typed node graph in which

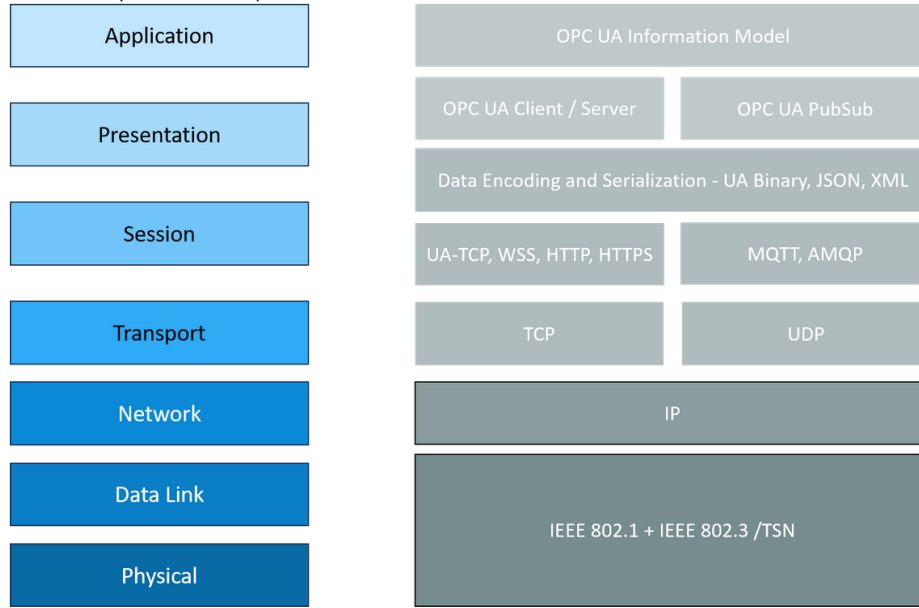


Figure 1: OPC UA OSI layer mapping. Source: Embien Technologies [14]

each node belongs to one of several node classes, including **Variable**, **Object**, **Method**, and **ReferenceType**, along with their typed relationships [10]. Clients traverse and access this address space using services such as **Read**, **Write**, **Call**, and **Browse**, all of which follow a synchronous request-response semantics at the service level.

The protocol stack for OPC UA client-server communication consists of three layers [15]. At the lowest level, standard TCP/IP provides the reliable, ordered byte-stream transport. Above this, the OPC UA TCP transport protocol (`opc.tcp://`) defines the framing and connection lifecycle, including the **Hello**, **Acknowledge**, and **OpenSecureChannel** messages used during connection establishment. At the highest layer, OPC UA binary encoding serializes all request and response structures into a compact binary format, minimizing message overhead and making the stack applicable in resource-constrained or latency-sensitive industrial environments [15].

Prior to invoking any application-level service, the client must establish a Secure Channel and a Session [13]. The Secure Channel is negotiated during the **OpenSecureChannel** handshake, in which both parties agree on a security policy that specifies the signature algorithm and, optionally, the symmetric encryption algorithm applied to all subsequent messages. The Session layer is established on top of the Secure Channel and carries application-level identity, including user authentication tokens. All service invocations are bound

to an active session.

Although the OPC UA client-server model is fundamentally request-response based, it includes an asynchronous notification mechanism to avoid the inefficiencies of repeated polling [13]. A client creates a Subscription on the server by invoking the `CreateSubscription` service, specifying a publishing interval that controls how frequently the server evaluates and reports changes. Within a subscription, the client registers individual Monitored Items via `CreateMonitoredItems`, each referencing a specific node attribute. The server samples each monitored item at the requested sampling interval and enqueues notifications when the monitored value changes beyond a configurable deadband. Accumulated notifications are delivered to the client in response to periodically issued `Publish` requests, which the client sends proactively to maintain a pipeline of outstanding requests [13]. This mechanism therefore operates within the client-server request-response framework at the transport level, despite providing event-driven semantics at the application level.

It should be noted that this subscription mechanism bears terminological similarity to the OPC UA Publish-Subscribe (PubSub) model defined in Part 14 of the specification. [3]. These two models are architecturally distinct. Throughout this thesis, the term *Client-Server* refers exclusively to the connection-oriented, session-based model described in this section, including its subscription and monitored-item services. The term *Publish-Subscribe* or *PubSub* is reserved for the Part 14 transport mode.

2.1.2 Publish-Subscribe Communication

In contrary to the session-based Client-Server model, the Publish-Subscribe (PubSub) model [3] decouples data producers from data consumers at the transport level. Publishers transmit data onto the network without establishing a direct connection to any receiver, and subscribers consume that data independently without initiating service calls toward the publisher [3]. A single publisher can thereby reach an arbitrary number of subscribers without any per-subscriber connection overhead.

OPC UA PubSub supports two categories of transport infrastructure [3]. In broker-based transport, messages are routed through a message broker, such as an MQTT [16] or AMQP [17] broker. Publishers send messages to a named topic on the broker, and subscribers receive messages by subscribing to that topic. The broker is responsible for message routing, queuing, and, depending on the protocol, delivery guarantees.

In brokerless transport, messages are transmitted directly over the network without any intermediate routing component. OPC UA defines bro-

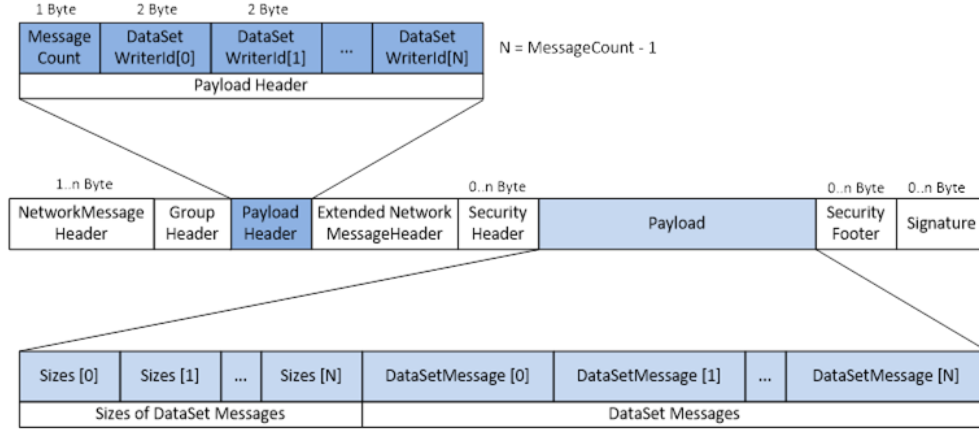


Figure 2: Structure of a single UADP-packet carrying multiple DataSetMessages

kerless transport over UDP, in which NetworkMessages are encapsulated in UDP datagrams and transmitted to an IP multicast group address [3]. A **Publisher** groups its data into one or more **WriterGroups**, each associated with a configurable transmission interval. Within each **WriterGroup**, one or more **DataSetWriters** serialize a **PublishedDataSet**, composed of individual **Fields**, into a **DataSetMessage**. As shown in Figure 2, multiple **DataSetMessages** are aggregated into a single **NetworkMessage** and serialized using the UA Datagram Protocol (UADP) encoding [3], which defines the binary layout of the datagram including the **NetworkMessage** header, **Group-Header**, and individual **DataSetMessage** payloads. The resulting datagram is then transmitted to the multicast group address. IP multicast allows this single datagram to be delivered to all hosts that have joined the corresponding multicast group [18], without requiring the sender to enumerate or address each receiver individually. The publisher sends one **NetworkMessage** per **WriterGroup** transmission interval, and the network infrastructure replicates the datagram to all group members at the routing level, avoiding redundant transmissions from the source. Upon receiving a **NetworkMessage**, a **DataSetReader** filters the message by **PublisherId**, **WriterGroupId**, and **DataSetWriterId**, and deserializes the relevant **DataSetMessage** fields into the subscriber's local address space.

2.2 Hardware-based I/O Virtualization with PCIe and SR-IOV

Virtualized environments require mechanisms to share physical I/O devices between multiple isolated instances. Since the number of containers or virtual machines typically exceeds the number of available hardware devices, direct one-to-one assignment is not sufficient [19].

2.2.1 Overview

I/O virtualization can be realized using three main approaches: emulation, paravirtualization, and hardware-based passthrough [19]. These approaches differ in how the virtualized instance interacts with the underlying physical device and in the resulting performance and flexibility trade-offs.

Emulation provides a fully software-based abstraction of a hardware device. The virtual machine interacts with a simulated device for which it can use standard drivers. All I/O operations are intercepted by the hypervisor or a trusted virtual machine and executed on the physical hardware using a trap-and-emulate mechanism [20]. This approach offers high compatibility, as no modifications to the guest system are required, but introduces significant overhead due to the additional software layers [19].

Paravirtualization reduces this overhead by replacing fully emulated devices with virtualization-aware interfaces [21]. Instead of simulating hardware, the guest system communicates with the host through optimized drivers and shared mechanisms. While this approach improves performance compared to emulation, it still relies on host-side processing and introduces additional overhead compared to native I/O [22].

Passthrough I/O Hardware-based virtualization removes most software layers by assigning a physical device directly to a virtual machine. This enables direct communication between the guest and the device, resulting in near-native performance and reduced CPU overhead [23]. However, passthrough does not allow sharing of a device between multiple instances, which limits scalability [24].

2.2.2 Hardware Support for Device Passthrough

In virtual machine environments enabling device passthrough for untrusted guests requires hardware support for memory isolation. Modern x86 systems

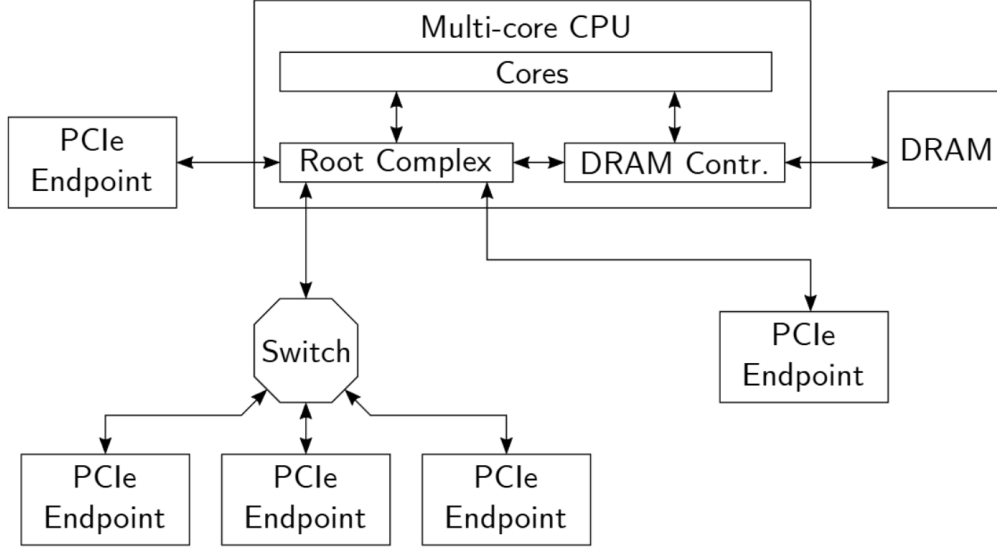


Figure 3: Example PCIe topology of a multi-core CPU with integrated Root Complex [19].

therefore include Memory management Units (MMUs) [25] and I/O Memory Management Units (IOMMUs) [26], for example Intel VT-d, AMD-VI. The MMU prohibits any VM to use their assigned CPU cores to read and write from any memory (RAM, I/O devices, etc.) that doesn't explicitly belong to them. IOMMU has the same task but for any I/O device's Direct Memory Access (DMA) engine. DMA operations can read and write from memory on the system and the IOMMU ensures that each I/O device can only use such operations on memory assigned to the Virtual Machine (VM) or container that is assigned to the device.

PCI Express (PCIe) PCIe [27] is the technology that connects passthrough I/O. It is an update to the PCI [28] and PCI-X [29] technology. In the topology in Figure 3 an example of the PCIe topology can be seen. The root complex is a single node that connects with the Cores and memory to the I/O subsystem. It can have multiple PCIe ports that connect either to PCIe devices (also called Endpoints) directly or to switches, which then connects to the PCIe Endpoints. This root complex has to be integrated in hardware.

PCIe functions and memory mapping For system components to address PCIe components, PCIe functions are introduced. Each function is independently addressable through the memory map of the system. The

mapping happens at the boot of the firmware or the OS. In x86-based systems, the BIOS maps memory by having each PCIe function request resources, reserving an address range in response, and returning the allocated space to the function.

Afterwards a function is reachable for CPU communication. A transaction follows the pattern of identifying the address of the function, compiling a packet and putting it on the PCIe interconnect. This method is called Memory Mapped I/O (MMIO). Hereby both DRAM and I/O devices addresses are saved in a single memory map, which makes machine instructions that are built for DRAM communication usable for MMIO transactions. This has the benefit that PCIe devices that support Direct Memory Access (DMA) can use the same memory map to access DRAM as well as other PCIe devices [19, 27].

2.2.3 Single Root I/O Virtualization (SR-IOV)

In the design of passthrough I/O a trusted software intermediary like a hypervisor is removed to enable the performance benefits. This leads to limitations in scalability, because it makes sharing a physical I/O device impossible as there is no software to manage it. This leads to hardware costs rising, when a VM is added to the system, because each need their own physical device. This limitation lead to research into self virtualizing I/O devices [24] [30]. The idea being to offload the performance penalties of software-based virtualization into the hardware. Devices implementing this concept can offer virtual interfaces on a physical function that can be addressed by VMS.

The PCIE Special Interest Group (PCI-SIG) released the Single Root I/O Virtualization and Sharing Specification [31] to implement these ideas. As the name suggests it is specified for PCIe topologies that utilize a single Root Complex. Designs for a multi root complex also exist [32]. This MR-IOV will however not be covered in this thesis.

Physical and Virtual Functions To offer SR-IOV, a device needs to provide at least one PCIe function that supports the SR-IOV capability. These functions are then called Physical Functions (PFs) and are supposed to be controlled by the trusted host in the system. The PF offers an interface that makes it possible to spawn multiple PCIe functions dynamically. These functions are then called Virtual Functions (VFs).

The system itself can't make up a difference between classic PCIe functions, PFs and VFs. The mapping for VFs stays the same, hence VFs and PCIe passthrough are automatically compatible. MMU and IOMMU guarantee hardware-enforced spatial isolation and manufactures of SR-IOV devices

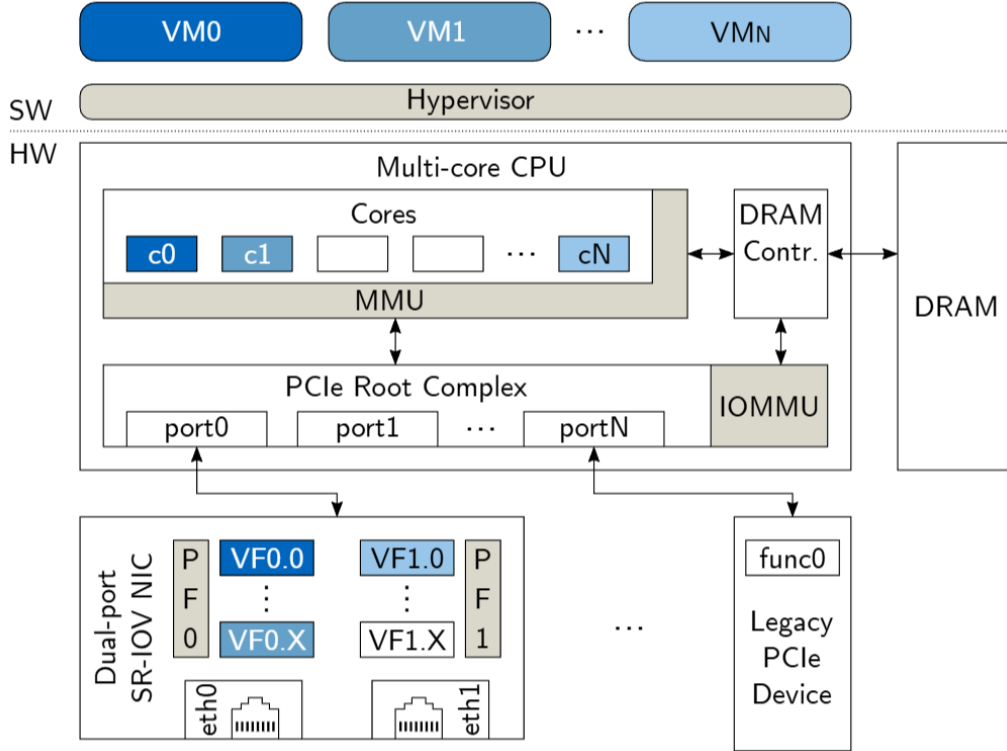


Figure 4: SR-IOV allows a single physical resource, such as an Ethernet port, to be shared across multiple virtual machines. In this example, both VM0 and VM1 are assigned to the same physical NIC port eth0 [19].

provide special VF drivers for the usage in VMs. A visualization of this process is in Figure 4. In practice, Virtual Functions are exposed to the host system as independent PCIe devices and can be assigned directly to virtual machines or containers. Once assigned, a VF provides a dedicated interface through which the guest system can interact with the underlying hardware. This includes transmitting and receiving data, as well as configuring device-specific parameters within the limits defined by the Physical Function [19, 31].

SR-IOV Operation Modes SR-IOV devices, particularly their network interface cards, can operate in different modes, commonly referred to as **legacy mode** and **switchdev mode** [33]. These modes differ in how switching and traffic management are handled.

In **legacy mode**, each Physical Function is paired with an embedded switch (eSwitch) per external LAN port, connecting all VF Virtual Station

Interfaces attached to that port [34]. The eSwitch operates autonomously, forwarding frames in hardware based on MAC address lookup with no configurable forwarding policy exposed to the host [35].

In `switchdev` mode, the driver exposes the eSwitch to the host kernel through the switchdev abstraction layer [33]. One port representor netdevice is created per VF, serving as the control-plane endpoint for that VF within the host networking stack. Traffic control rules written against these representors can be offloaded into the hardware datapath of the eSwitch, allowing forwarding behavior to be explicitly programmed rather than autonomously determined by the device [34].

2.3 Linux Traffic Control

The Linux Traffic Control (TC) subsystem provides mechanisms to control how packets are processed, scheduled, and transmitted within the Linux networking stack [36]. It can operate both on the egress and ingress path of the network interfaces, allowing inbound and outbound traffic to be filtered and policed [37].

Traffic Control is used to implement policies such as rate limiting, traffic shaping, prioritization, and packet filtering [36]. These mechanisms are required to manage bandwidth usage, reduce congestion, and enforce quality of service (QoS) in networked systems. Additionally, they can be used to enforce traffic isolation and sorting.

The functionality of Traffic Control is based on queueing disciplines (Qdiscs), which define how packets are enqueued, scheduled, and transmitted [37]. Classful Qdiscs can contain classes, and each class can be assigned its own child Qdisc, enabling hierarchical traffic management structures. Filters are attached to Qdiscs or classes to classify packets into specific processing paths based on header fields or other criteria [37].

2.3.1 Queueing Disciplines (QDisc)

Traffic Control configures queueing disciplines on network interfaces to determine how packets are handled before transmission. By default, packets are sent using a simple first-in-first-out strategy. More advanced queueing disciplines allow control over packet flow through mechanisms such as buffering, prioritization, rate limiting, delays, or packet loss [38, 39].

Filters are attached to queueing disciplines to classify incoming packets and trigger actions based on matching criteria. On the ingress path, an ingress qdisc is attached to the interface, and filters are applied against it to intercept packets as they arrive. When a packet matches the filter criteria,

an associated action is executed, such as redirecting the packet to another interface, dropping it, or passing it through unmodified [36].

The ingress path is relevant in this work because the Traffic Control rules do not shape or schedule traffic, but instead use filters to classify and redirect packets between interfaces. Each filter is evaluated against arriving packets, and matching packets are acted upon immediately, without entering any egress queue [36, 39].

2.3.2 TC Flower

TC Flower is a flow-based classifier within the Linux Traffic Control subsystem that matches packets against header fields and triggers corresponding actions. Unlike simpler classifiers, Flower can match on a wide range of fields simultaneously, including VLAN tags, Ethernet source and destination addresses, IP addresses, transport protocol, and port numbers. [36, 39]

In this work, Flower is used on the ingress path to match packets by VLAN ID and destination MAC address, and to redirect them between physical interfaces. The classifier is evaluated per arriving packet, and only packets satisfying all specified match criteria trigger the associated redirect action.

Where supported by the network interface card, Flower rules can be offloaded to hardware, allowing matching and actions to execute directly on the device. This reduces CPU involvement and is particularly relevant in SR-IOV and high-throughput networking setups. [39]

2.3.3 NAPI

NAPI is the event handling mechanism used by the Linux networking stack, originally introduced as New API in Linux 2.4 but no longer treated as an acronym. It abstracts interrupt and polling-based packet processing away from individual drivers, allowing incoming and outgoing packets to be handled uniformly across different hardware [40].

When a packet arrives, the NIC demodulates the signal and stores it in device memory, from where it is transferred via the RX DMA engine into the Rx ring buffer in kernel memory. NAPI then schedules processing of these packets, passing them up through the network stack including TCP and UDP processing, until the data is copied into userspace via the socket queue [40].

For Traffic Control, the most relevant part of this path is the queueing discipline, which sits on the interface and determines how packets are handled before they are passed further up or transmitted out.

2.4 Network Isolation via VLAN

A Virtual Local Area Network (VLAN) is an abstraction of the Local Area Network that provides data link layer connectivity to a subnet without requiring physical changes to the network infrastructure. By assigning switch ports or devices to specific VLANs identified by a unique VLAN ID, traffic can be confined to a defined group of hosts. Devices within the same VLAN communicate as if they were connected to the same physical network, regardless of their actual physical location [12, Clause 5].

This isolation is enforced by Ethernet switches, which associate each port with one or more VLAN identifiers. Frames traversing the network are tagged according to the IEEE 802.1Q standard, which inserts a 4-byte field into the Ethernet frame between the source MAC address and the EtherType field. This tag consists of a 2-byte Tag Protocol Identifier (TPID) and a 2-byte Tag Control Information (TCI) field. The VLAN ID is encoded in a 12-bit subfield of the TCI, allowing up to 4,094 usable VLAN identifiers, with values 0 and 4095 reserved [12, Clause 9].

Because inserting the VLAN tag modifies the frame, 802.1Q encapsulation requires a recalculation of the frame check sequence. The IEEE 802.3ac standard accordingly increased the maximum Ethernet frame size from 1518 bytes to 1522 bytes to accommodate the additional four bytes [41].

VLAN-enabled ports are generally categorised as either tagged (trunk) or untagged (access). Trunk ports carry traffic from multiple VLANs simultaneously and are typically used to interconnect switches, whereas access ports carry traffic for only a single VLAN and connect to end devices. When a frame arrives on an untagged access port, the switch associates it with a VLAN based on the port configuration and inserts an 802.1Q tag for internal processing and transmission over tagged links. Frames received on trunk links are already tagged and are forwarded based on their VLAN identifier. Upon egress at an access port, the tag is stripped before delivery to the host, keeping the VLAN infrastructure transparent to end devices [12, Clause 8].

Each VLAN constitutes its own broadcast domain. Unicast, broadcast, and unknown unicast frames are flooded only within the ports belonging to the same VLAN, preventing broadcasts from propagating across VLAN boundaries and improving bandwidth utilisation by reducing unnecessary traffic [12, Clause 8].

2.5 Namespaces

Linux namespaces are kernel primitives that provide isolation by creating separate instances of global resources. For networking, this is implemented

via network namespaces.

Each network interface (e.g., physical NICs, virtual interfaces, or SR-IOV virtual functions) belongs to exactly one network namespace, and each process is associated with exactly one network namespace. A process can only access the interfaces and routing table within its namespace. As a result, each namespace forms an independent network stack.

This mechanism enables containerized workloads to use isolated networking contexts. In particular, NIC resources such as SR-IOV virtual functions can be assigned to specific namespaces, providing direct and isolated network access [42].

2.6 Containerization via Docker

Docker is a container runtime that executes applications in isolated environments on a shared operating system kernel. A container consists of a process together with its filesystem, configuration, and resource constraints. Isolation is implemented using Linux kernel features, primarily namespaces and control groups. Namespaces define separate views of system resources, while control groups limit and account for resource usage [43, 42].

For networking, Docker relies on Linux network namespaces. Each container is assigned its own network namespace, which contains an independent network stack including network interfaces, IP addresses, routing tables, and firewall rules. Processes inside the container operate only within this namespace and cannot directly access the network configuration of the host or other containers [42, 44].

In its default configuration (bridge mode), Docker connects containers to the host network by creating a virtual Ethernet pair. One end of the pair is placed inside the container's network namespace as a network interface, typically named `eth0`. The other end remains in the host namespace and is attached to a virtual bridge, usually `docker0`, which connects multiple containers on the same host at Layer 2 [43].

Each container receives an IP address within the subnet assigned to the bridge network. Communication between containers on the same bridge is handled via the bridge device, while communication with external networks is routed through the host. Docker configures Network Address Translation (NAT) rules using `iptables` so that outgoing traffic from containers is translated to the host's IP address. Incoming connections can be forwarded to a container by binding a host port to a container port, creating a rule that directs traffic arriving at the host interface to the corresponding container IP and port [43].

While the bridge-based setup described above represents Docker’s default networking mode, it is not the only available configuration. Docker also supports alternative modes such as host and none networking, and more generally builds on standard Linux networking primitives. Consequently, networking can be configured independently of Docker by directly manipulating network namespaces and interfaces. This approach provides greater control over interface placement and connectivity, and is the method used in this work [43].

Network interfaces can also be manually moved into a container’s network namespace using standard Linux tools such as `ip link set netns` [45]. For this a symlink needs to be established between the docker namespace and the kernel namespace with

```
ln -sfT /proc/<container_pid>/ns/net
/var/run/netns/<ns_name>
```

, where `container_pid` is the process ID of the container and `ns_name` is the name of the namespace that is then accessible with `ip link` commands. Since Docker relies on native Linux namespaces, these namespaces can be inspected and manipulated using tools such as `ip netns` or `nsenter`, like any other Linux network namespace.

2.7 Integration

To achieve hardware-accelerated, isolated, and traffic-shaped container networking, the three concepts of SR-IOV, custom qdiscs, and VLAN tagging are combined into a unified setup, in which each component builds upon the previous to form a cohesive network architecture.

2.7.1 Integration of SRIOV with docker environments

Giving each container a I/O device through SR-IOV can be achieved by using a Network plugin or direct assignment through Linux namespaces. The networks plugins offer little control but are easily manageable while the direct assignment needs their own orchestration script, especially for parameterized environments.

Network plugin Cluster orchestrators like Kubernetes offer SR-IOV plugins that manage the configuration and VF assignment on their own. Examples include Intel® SR-IOV Container Network Interface (CNI) Plugin [46], Multus CNI [47], SR-IOV Network Device Plugin for Kubernetes [48], and

Kuryr-Kubernetes [49], but others are possible as well. Using these plugins can however have drawbacks. Relying on third-party abstractions and automated approaches can reduce control over resources [50], flexibility [51] and specific optimizations [52, 53]. In addition, compatibility issues with the hardware can arise [54].

Direct Assignment A more manual approach assigns VFs directly to containers by leveraging Linux network namespaces, as described in Sections 2.6 and 2.5. After the docker container is linked to a linux namespace and is accessible as `<container_ns>`, a VF interface `vf_interface` can then be moved into it by using standard Linux tooling

```
ip link set <vf_interface> netns <container_ns>
```

[45]. Once inside the namespace, the VF is exclusively accessible to the container’s processes and invisible to the host or other containers. No virtual bridge or additional switching layer is required, as the hardware eSwitch on the NIC handles inter-VF traffic. This provides the container with near-native network performance, since the data path between the container and the physical network traverses no software virtualization layers [55]. This is the approach adopted in this work.

2.7.2 Applying Queueing Disciplines in SR-IOV

Attaching queueing disciplines to VF traffic requires the NIC to operate in switchdev mode, as the representor netdevices created in that mode serve as the attachment point for `tc` rules within the host networking stack. Qdiscs and filters attached to a representor apply to all traffic associated with the corresponding VF.

If the NIC driver supports hardware offloading and the `hardware-tc-offload` feature is enabled, compatible TC rules are translated into hardware flow entries inside the eSwitch pipeline, executing packet classification and forwarding actions entirely on the device [56].

2.7.3 Implementing VLANs in SR-IOV

VLAN isolation in SR-IOV environments can be realized either directly in hardware by the NIC or in software within the Linux networking stack. Both approaches differ in where tagging and filtering are performed and in their impact on performance and flexibility [57].

Implementation in Hardware NICs - SR-IOV SR-IOV extends VLAN isolation below the hypervisor layer by implementing it directly in the NIC hardware. VLAN membership is enforced at the VF level by the PF, without involving the host networking stack in the data path.

In both legacy mode and switchdev mode, a VLAN ID can be assigned to a VF using the `ip link set` command, for example

```
ip link set enp1s0f1np1 vf 0 vlan 100
```

[58]. The NIC then inserts and strips IEEE 802.1Q tags transparently for all traffic associated with that VF.

In switchdev mode, VLAN tagging and filtering can alternatively be implemented using TC flower rules attached to QDiscs. Depending on the capabilities of the NIC and driver, these rules may be offloaded to hardware. If offloading is not supported, the processing is performed in software [57].

The Intel e810 used in this thesis is not able to offload VLAN tagging and stripping via tc flower in hardware, necessitating another approach [34].

Implementation in Software As an alternative, VLAN functionality can be implemented in software by creating a VLAN netdevice on top of an existing interface, for example with `ip link add link enp1s0f1v1 name enp1s0f1v1.100 type vlan id 100` [45]. The VLAN device acts as an upper interface associated with a lower physical or virtual interface.

Outgoing packets transmitted through the VLAN device are encapsulated with an IEEE 802.1Q header before being passed to the lower device. Incoming tagged frames received on the lower device are delivered to the corresponding VLAN device, where the VLAN header is processed in the kernel [12].

This processing is integrated into the Linux networking stack and does not require packets to leave and re-enter the kernel solely for VLAN handling. NAPI is responsible for efficient [40] at the driver boundary, while VLAN tagging and stripping are handled in the kernel transmit and receive path above the device driver [36].

Compared to hardware-based approaches, software VLANs provide greater flexibility but introduce additional processing overhead, as all tagging and filtering operations are executed by the CPU. This has the advantage that VLAN tagging and stripping is the first/last step in the network meaning that the TC-flower rules, which rely on the availability of the VLAN tag, can be offloaded in hardware.

3 Related Work

The following section reviews existing work related to OPC UA communication models, their deployment on resource-constrained hardware, and the use of network virtualization technologies such as SR-IOV. Prior research has investigated the performance characteristics of OPC UA client-server and PubSub architectures, as well as their suitability for real-time industrial applications. This section summarizes these contributions and identifies limitations in current approaches, which motivate the design of the proposed testbed.

3.1 OPC UA Communication Models

Burger et al. [59] perform systematic performance measurements of the OPC UA client-server and broker-less pub/sub models on resource-constrained hardware. CPU utilization is identified as the primary bottleneck in both cases. On a Raspberry Pi Zero, the pub/sub publisher sustains up to 40 000 signals per second before CPU saturation, while memory consumption remains below 15 MB and network utilization below 6 Mbps across all configurations. Client-server throughput degrades sharply once more than 20 concurrent clients share a single server, an effect attributed to per-session management overhead. The two models thus exhibit fundamentally different scaling behavior with respect to node count.

Pfrommer et al. [60] implement broker-less OPC UA PubSub over TSN using the open62541 SDK. A non-realtime OPC UA server and a hardware-interrupt-driven PubSub publisher share a common information model without sacrificing real-time guarantees. Latency measurements under Linux with **RT-Preempt** patches yield sub-millisecond publication intervals at 50 ns jitter, demonstrating the viability of deterministic publish cycles on commodity hardware.

Arestova et al. [61] extend this evaluation to off-the-shelf hardware without proprietary software stacks, measuring OPC UA PubSub over TSN under multiple Linux queuing disciplines. Credit-Based Shaping yields jitter of $\pm 250 \mu\text{s}$; Time-Aware Shaping reduces round-trip times to approximately 800 μs at moderate payload sizes. Worst-case latency bounds are derived independent of any specialized hardware, confirming that PubSub traffic can coexist with standard TSN infrastructure.

3.2 Building Testbeds using Network Virtualization and SR-IOV

Lantz et al. [8] introduce Mininet, a network emulator that instantiates lightweight Linux containers on a single host, connected via virtual Ethernet pairs and software switches. Arbitrary topologies can be prototyped at low cost, but Mininet provides no performance isolation: virtual hosts share CPU and I/O resources without guarantees, making timing measurements unreliable. Handigol et al. [5] address this with Mininet-HiFi, adding CPU bandwidth limiting and link emulation. Fidelity nonetheless degrades at link rates above 100 Mb/s and deteriorates further as concurrent host count increases. Emmerich et al. [6] quantify the root cause: moving from physical NICs to Open vSwitch reduces throughput by up to 90%, driven by context switching, packet copying, and virtual device emulation overhead. These results establish a hard performance ceiling for software-only emulation approaches.

SR-IOV was then used to address these overheads. Liu et al. [62] demonstrate that this approach sustains near line-rate throughput across 60 VMs with negligible CPU overhead. Wiedner et al. [7] apply SR-IOV to virtual testbed construction in HVNet, using hardware timestamping on every packet across 100 randomly generated topologies. The resulting latency and jitter distributions are up to three orders of magnitude more accurate than Mininet. Gallenmüller et al. [4] consolidate an entire testbed onto a single SR-IOV-accelerated server, confirming substantially higher throughput stability over purely software-based virtualization. Taken together, these results establish SR-IOV as the enabling mechanism for high-fidelity single-host testbeds, motivating its adoption in the present work.

3.3 This Work Differs

HVNet [7] and TOAST [4] validate SR-IOV-based virtual testbeds against synthetic traffic benchmarks. Neither addresses application-layer protocol behavior, nor evaluates whether timing fidelity is preserved at the level of granularity required for industrial protocol selection.

This work makes three concrete contributions. First, the testbed is deployed entirely in Docker containers rather than bare-metal VMs, directly testing whether the SR-IOV hardware I/O path remains accessible and functional under container-based network namespacing. Second, the testbed is a unique approach having one PC handling the applications and another handling the network, while traffic between them is offloaded in hardware as much as possible. Third, the testbed is used to conduct a systematic per-

formance comparison of OPC UA PubSub and Client-Server under varying publisher counts and network loads, using the same Raspberry Pi hardware as the physical reference. This directly answers whether an SR-IOV virtual testbed reproduces latency and throughput characteristics at sufficient fidelity to inform deployment decisions between the two communication models.

Acknowledgements

This work was carried out in collaboration with and funded by AXSOL GmbH [63]. The author would like to thank Amer Durakovic, Simo Kauth and Alexej Grigorjew for their support and guidance throughout the project.

During the writing of this thesis, large language model-based tools were used to assist in formulating and revising sentences. All technical content, results, and conclusions are the author's own.

4 Methodology

This chapter describes the methodology employed for the design, implementation, and operation of the testbed, as well as the procedures used for measurement generation. Initially, the construction and configuration of the SRIOV and Raspberry Pi testbed are detailed, followed by a description of the software used for traffic generation and lastly the methods of extracting, analyzing and evaluating data are discussed.

4.1 Testbed

This chapter outlines the hardware, software and architecture employed for the design of the testbed.

4.1.1 Hardware

The experimental testbed consists of two dedicated x86-based systems, referred to as the host node and the switch node. Both systems are based on the Intel Raptor Lake platform and provide PCI Express connectivity for high-performance networking components.

Network Interfaces Each system is equipped with an Intel Ethernet Controller E810-C (revision 02) [34] installed as a PCI Express device. The adapter is a QSFP+ based variant supporting up to 100 Gigabit Ethernet (100 GbE) per port. Each controller exposes two Ethernet ports.

SR-IOV functionality was then enabled on both systems. In preparation the ice driver and corresponding kernel headers needed to be installed from the official source repository [58]. Following the driver initialization, the Ethernet Port Configuration Tool (EPCT) [64] was used to configure the port layout of the adapter. The controller was set in 2x100 GbE mode. After applying the configuration, the driver module was reloaded and the systems were rebooted to activate the changes.

Data Path The cards are interconnected via a multimode fiber optics 100 GbE cable. For the experiments only one physical port per system was utilized, as they did not require usage of more than 128 virtual functions. However with small modifications the software logic can be applied to more ports and virtual functions as well. Both systems were hosted in a controlled laboratory environment and accessed remotely via the university’s virtual private network. Experimental data traffic was transmitted exclusively over the dedicated 100 GbE.

4.1.2 Software

All software components were developed and deployed remotely via SSH access to both nodes.

The application layer was built on the open-source library `open62541` [65], which was used to implement a custom OPC UA server configurable at startup as either a publisher or subscriber in the PubSub model, or as a server or client in the client-server model. To ensure reproducibility and consistent deployment across both nodes, the server was containerized using Docker [43], with the source code versioned and distributed via Git [66].

Each container was assigned its own network namespace, providing hardware-level isolation of the network datapath between application instances. For each container, a virtual function was created and moved into the corresponding namespace. To automate and parameterize this process, a Python orchestration script was developed. The script reads a central `.toml` configuration file, which serves as the single source of truth for all deployment parameters, and uses it to provision the virtual functions, configure the network interfaces, and bring up the containers in the correct order.

4.1.3 Network Architecture

The architecture was designed to leverage SR-IOV hardware capabilities to achieve near bare-metal network performance while maintaining complete datapath isolation between all traffic flows. Three successive approaches were developed toward this goal, each informing the next, culminating in the final solution. The first approach uses the `legacy` mode SR-IOV with network isolation via the VLAN handling of the embedded switch. Because the embedded switch of the particular network card used does global mac learning and not learning per VLAN ID, the approach failed and the second one was developed using layer 3 routing. However the aims of the thesis, to design a testbed that does most switching logic hardware offloaded, were not sufficiently reached in this approach and the routing tables got exceedingly more complicated for each new container. This necessitated the third approach using the `switchdev` mode of SR-IOV. Software VLAN links were used to negate the mac learning issue of the first approach bypassing the VLAN handling of the embedded switch. The design also doesn't use layer 3 routing, keeping it simple and scalable. All three were initially designed around a TCP unicast structure and the last one was then extended to support multicast.

Approach 1 — Legacy Mode In the first approach, the physical functions were left in `legacy mode`, which supports hardware-offloaded VLAN tagging and stripping at the embedded switch. On the host, each container’s network namespace was first registered in the Linux namespace registry (`/var/run/netns/`) via symlink, making it visible to standard `ip netns` tooling. To achieve datapath isolation, traffic from each container on the host node is tagged with a unique VLAN ID before being forwarded to the switch node. The switch node mirrors the virtual function layout of the host node, with each VF paired to its counterpart. These VFs are then added to a software-implemented Layer 2 bridging topology. The complete setup is shown in Figure 5, which in theory should guarantee fully isolated packet paths.

In practice, however, communication between two containers was impossible due to the NIC’s internal switching logic. As illustrated in Figure 6, a reply packet from the subscriber carries a destination MAC address that the NIC recognizes. When this packet reaches the embedded switch, the switch attempts to deliver it directly to the known destination. Since the packet does not belong to the destination’s VLAN, it is dropped and never forwarded to the switch node. This behavior causes any bidirectional communication to fail, necessitating a different approach. Note that the underlying issue for this is the hardware design using a global MAC learning table. If a table for each VLAN is used, as it might be the case in different network cards, this design works.

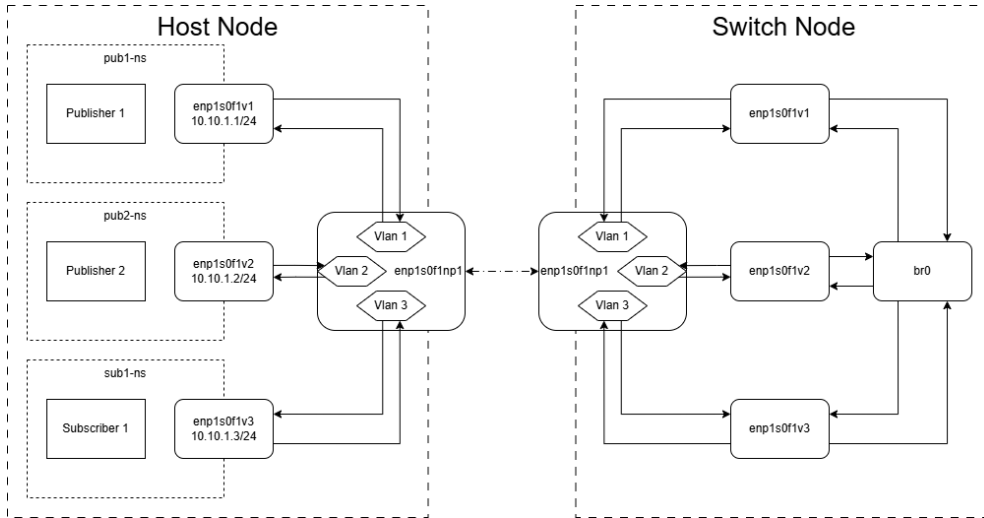


Figure 5: First iteration of the testbed.

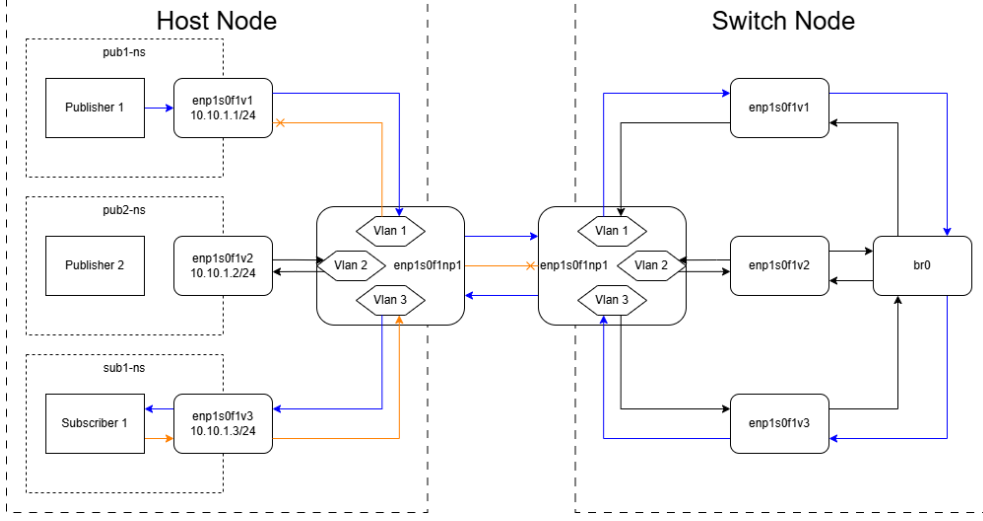


Figure 6: Simplified dataflow of an ARP request. The ARP reply is dropped at `enp1s0f1np1` of the host node because the embedded switch recognizes the destination MAC but the packet does not belong to its VLAN.

Approach 2 — Routing The next solution was designed to mitigate the issues with the NIC throwing away packets. The root cause of the previous failure was that the embedded switch maintained a global MAC learning table: when a reply packet arrived, the switch recognized the destination MAC address and attempted to deliver it directly to the known virtual function, bypassing the VLAN boundary and dropping the packet in the process. To circumvent this, layer 3 routing was introduced between the containers. By routing traffic through an intermediate hop, the destination MAC address of any given packet is no longer that of the final recipient, but rather that of the next-hop router interface. The embedded switch therefore never encounters a MAC address it can directly resolve to a local virtual function, forcing all traffic to leave via the physical port as intended. An example configuration with two publisher and one subscriber container is shown in Figure 8.

Each VF is assigned a unique IP address within its own /24 subnet (10.10.x.1/24). A route into the aggregated 10.10.0.0/16 subnet is also configured, with the next hop pointing to the corresponding switch-side address. The switch node mirrors this structure with an equal number of namespaces and VFs. Each switch-side VF resides in its own dedicated namespace and acts as the counterpart to the corresponding host-side VF, addressed within the same /24 subnet at 10.10.x.254/24. Each switch namespace is then connected to the root namespace (namespace 0) via a virtual Ethernet (`veth`) pair. Within the root namespace, the desired application topology is

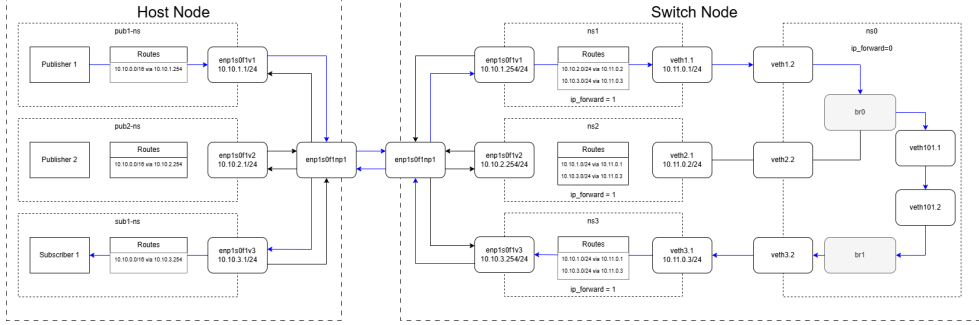


Figure 7: Sample data flow from publisher 1 to subscriber 2 with two bridges.

realized by bridging these `veth` interfaces. Each connecting `veth` endpoint is assigned an address in the connector subnet (10.11.0.x/24). Finally, per-namespace routes are installed directing traffic bound for 10.10.x.0/24 through the topology, using the corresponding `veth` address (10.11.0.x) as the next hop.

Figure 7 shows a sample data flow for a packet coming from publisher 1 to subscriber 1. A packet with the source of 10.10.1.1 and the destination 10.10.3.1 has following travel path. First in the namespace on the host it is recognized that the packet is in the 10.10.0.0/16 subnet and the initialized route sends it to the `next_hop` of 10.10.1.254. The packet arrives at the virtual function in the namespace 1 on the switch side. The defined route recognizes the packet as being in the 10.10.3.0/24 subnet and looks up the route which defines the `next_hop` as being 10.11.0.3. Because `ip_forward` is enabled the packet gets sent via `veth1.1` and `veth1.2` into the namespace of the topology, where it floods and finds the ip address 10.11.0.3 at `veth3.1` in the namespace 3. Here the packet is sent back to host node via the virtual function `enp1s0f1v3`, because `ip_forward` is enabled.

This method worked but was dropped because it was too complicated to configure with $n - 1$ routes being needed for n containers on the switch node. Also because of the switching issues described in approach 1 a Layer-2 network isolation via VLAN couldn't be achieved, leaving that part of the design to Layer-3 routing. In addition, multicast needed an entire separate architecture. These factors lead to the development of the third and final approach.

Approach 3 — Switchdev Mode The third approach uses Layer 2 switching again, but departs from approach 1 by placing the SR-IOV NIC into `switchdev` mode. Unlike the default SR-IOV embedded switch behavior, `switchdev` does not perform any autonomous forwarding decisions. Instead,

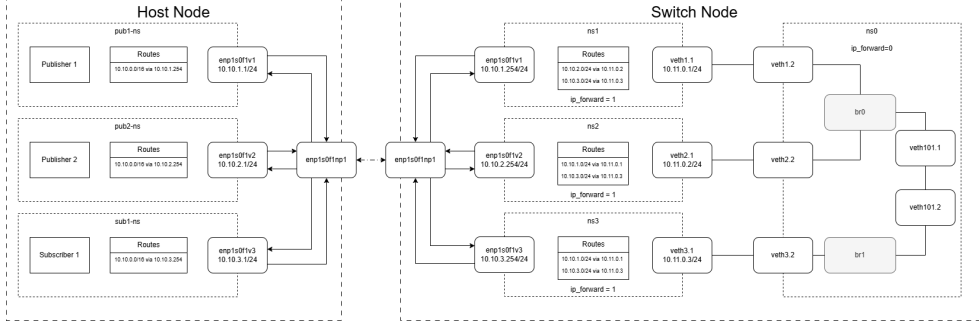


Figure 8: Sample setup of the switch with 2 bridges and 3 virtual functions

every steering rule must be provisioned explicitly via the kernel’s Traffic Control subsystem. In legacy mode, destination port selection precedes VLAN membership validation, causing the NIC to drop packets whose destination MAC resolves to a port outside the packet’s VLAN. The revised approach inverts this order by selecting the destination interface based on VLAN ID first, with each VLAN mapped to a dedicated software interface. The NIC acts solely as a distributor, forwarding packets to the interface corresponding to their VLAN tag.

Hardware offloading is enabled throughout with the call `ethtool -K enp1s0f1np1 hw-tc-offload on`, which means that once a matching TC flower rule exists, the NIC’s embedded switch forwards frames entirely in hardware without re-entering the kernel. The consequence of operating with hardware offload active is that VLAN stripping and tagging cannot be performed by the NIC itself. This function must therefore be either handled with a Qdisc rule in software or as used in this testbed with VLAN interfaces

The physical topology remains unchanged and on both nodes the Qdisc and TC flower configuration is structured identically around the physical function and its associated representor ports as can be seen in Figure 9. On the PF’s ingress path, a dedicated flower rule exists for each VLAN tag in use. Traffic arriving tagged with VLAN 1 is redirected to representor 1, traffic tagged with VLAN 2 is redirected to representor 2, and so forth. The rules on the representor side are deliberately simple, so that all egress traffic from a given representor is unconditionally redirected to the PF, since by the time a frame reaches a representor it has already been tagged with its VLAN ID and requires no further classification.

In code the rule for the representor 1 and VLAN tag 1 is:

```
tc filter add dev enp1s0f1r0 ingress \
    protocol all flower \
    action mirrored egress redirect dev enp1s0f1np1
```

sending all traffic to the physical function. On which the rule

```
tc filter add dev enp1s0f1np1 ingress \  
    protocol 802.1Q flower \  
    vlan_id 1 \  
    action mirrored egress redirect dev enp1s0f1r1
```

is added to send packets with the corresponding VLAN tag to the representor.

Each representor is the kernel-visible peer of a corresponding virtual function. The VF netdevice is in turn linked to a VLAN sub-interface created in Linux software. It is at this layer that VLAN tagging and stripping takes place, compensating for the hardware offload limitation noted above. Each VLAN sub-interface is assigned a distinct IP address following the scheme 10.10.1.x in the subnet 10.10.1.0/24, where x corresponds to the VLAN tag, yielding one addressable logical link per simulated network segment.

On the host node, each VLAN sub-interface is moved into the network namespace of the corresponding Docker container, giving the container exclusive ownership of that interface and its address. On the switch node the same interface hierarchy is built, but the VLAN sub-interfaces are attached to a Linux bridge rather than handed to individual namespaces, allowing the switch node to perform Layer 2 forwarding between segments.

This arrangement produces complete datapath isolation between tenants. Because each VLAN maps to its own VF, its own representor, and its own namespace or bridge port, there is no shared forwarding state between segments anywhere in the path. A frame belonging to VLAN x cannot, by construction of the flower rules, be delivered to the representor or VF of VLAN y. The result is a faithful emulation of a physically segmented network over a single shared 100 Gbit/s wire.

An example of a Request/Reply packet datapath can be seen in Figure 10, where the packet first gets sent from `publisher 1` to `vlan1.enp1s0f1v1` inside the docker namespace. Here the packet gets tagged and then sent over to the virtual function `enp1s0f1v1`. From there, the packet enters the embedded hardware switch inside the NIC, where it is matched against the TC flower rules that were installed via the representor `enp1s0f1r1`. Since the rule matches all traffic and forwards the packet to the uplink port `enp1s0f1np1`, the packet is forwarded entirely in hardware without surfacing to the host kernel. It then egresses through `enp1s0f1np1` towards the switch nodes physical function.

Here the `enp1s0f1np1` matches the VLAN ID 1 to the representor `enp1s0f1r1`, which then means the packet gets sent to `enp1s0f1v1`. Note here that the traffic is still tagged with VLAN 1 and only gets stripped when it is sent over to `vlan1.enp1s0f1v1`. This interface is then linked up to `br0` and after MAC

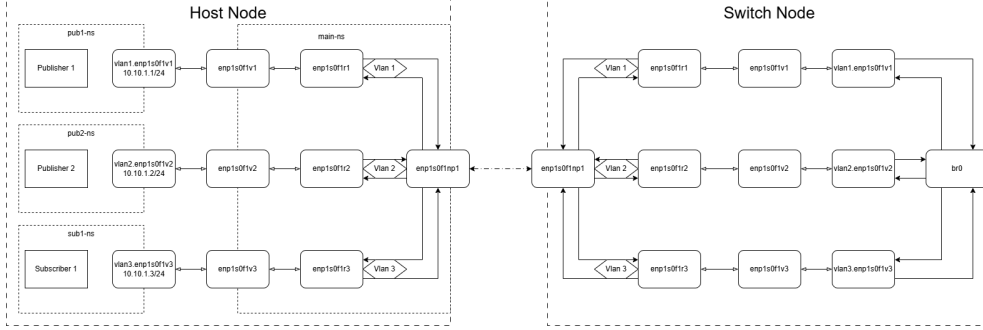


Figure 9: Sample setup of the switch with 2 bridges and 3 virtual functions

learning, is sent over to the next hop of `vlan3.enp1s0f1v3`. Here the same process starts again and the rule at the representor `enp1s0f1r3` redirects all traffic from `enp1s0f1v3` to the physical function. As all the traffic is coming from `vlan3.enp1s0f1v3` it is tagged with VLAN 3. This tag then gets picked up on the host node by the physical function as the rule redirects it to `enp1s0f1r3` and as such the packet appears at `enp1s0f1v3` and then gets transported to and VLAN stripped at `vlan3.enp1s0f1v3` of the host node. Lastly the packet arrives inside the docker namespace and at the desired IP.

The Reply then follows the same path back and because all rules are the same for all interfaces the switching logic stays the same as well. For multicast the only difference is that no MAC learning takes place and the network is flooded by the software bridge `b0`.

From a performance standpoint, the switchdev/offload architecture ensures that the kernel network stack is traversed exactly once in each direction. When a container transmits a packet, the frame passes through the kernel's transmit path on the VF netdevice, which includes software VLAN tagging on the sub-interface, and is then handed to the NIC. From that point onward, the embedded switch matches the frame against the pre-installed TC flower rules entirely in hardware and forwards it out of the physical port without any further kernel involvement. On the receive side, the NIC classifies the incoming frame by VLAN tag, steers it to the correct VF via the PF ingress rules, and the kernel processes it once through the receive path before delivering it into the target container's namespace.

4.1.4 Custom Topology

One of three logical topologies is instantiated at runtime on the switch node. The deployment script creates the required Linux bridges, attaches the VLAN-backed virtual function interfaces to those bridges, and intercon-

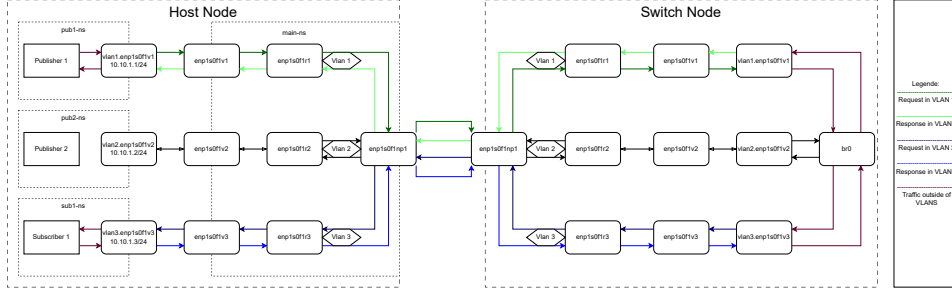


Figure 10: Example of a Request-Reply flowchart in the switchdev testbed.

nects bridges where required through `veth` pairs. The physical datapath remains unchanged. Only the software-defined forwarding structure on the switch node is varied.

In the star topology, all virtual functions are attached to a single bridge. This yields a single shared Layer 2 domain with one forwarding element and no intermediate hops between participants inside the emulated network.

In the line topology, each virtual function is placed on its own bridge and adjacent bridges are connected sequentially. Traffic therefore traverses a deterministic chain of intermediate forwarding steps, making path length dependent on endpoint placement.

In the tree topology, multiple leaf bridges aggregate sets of virtual functions and are then connected upward through additional parent bridges until a single root remains. This produces a hierarchical forwarding structure that scales beyond a flat bridge without requiring every participant to share the same local broadcast domain.

Other more complex topologies could also be customized, but are beyond the scope of this thesis.

4.2 Raspberry Pi Testbed

The physical testbed used for application deployment consists of four Raspberry Pi 3B+ [67] nodes, each based on the ARM64 architecture. All nodes are connected to a MikroTik switch within a shared subnet (10.0.0.0/24), forming a flat Layer 2 network. The host node of the SR-IOV testbed is connected to the same switch and subnet, making it reachable by all Raspberry Pi nodes.

Building the Docker image directly on the Raspberry Pi nodes was not feasible, as the build process consistently overloaded the devices due to their constrained computational resources. Instead, the SR-IOV host node was used to cross-compile and build the Docker image targeting the `linux/arm64`

platform. The resulting image was subsequently pushed to a private container registry, from which the Raspberry Pi nodes could pull it.

Each Raspberry Pi node pulled the pre-built Docker image from the registry and instantiated the application container locally. This approach decoupled the resource-intensive build step from the lightweight pull-and-run step performed on the constrained nodes.

Upon completion of each run, the collected log files were transferred from the Raspberry Pi nodes to the analysis laptop via `scp` over SSH for offline post-processing.

4.3 OPC UA Application Implementation

A custom OPC UA application was developed using the `open62541` C library [65]. The application is fully configuration driven using a TOML file as the single source of truth for the system data model. For this TOML file a generator script was written, to change the configuration based on a few parameter inputs, namely `publisher_count`, `group_count`, `dataset_count`, `field_count` and `interval`.

The application supports four operational modes namely `publisher`, `subscriber`, `server` and `client`, which it reads from the `role` set in the environment. The transport protocol is also configurable via environment variable with the options being `TCP`, `UADP` and `MQTT`, while the latter was not used in the evaluation due to bugs during development and time constraint of the thesis.

For the evaluation, one subscriber/client collects data from every publisher/server, because of the focus of this work being the evaluation of the network architecture. With small configuration of the config files however multiple subscribers/clients and a custom data selection per subscriber/client can also be realized.

In addition, every application exposes a TCP endpoint, which can be accessed via a tunnel to the host node. A third party application like `ua expert` [68] can then visualize the server and also the changes in data.

4.3.1 Pub/Sub using UDP

In UDP mode, each publisher instance establishes a single UDP multicast connection to `opc.udp://224.0.0.22:4840/` and constructs its PubSub topology directly from the TOML configuration, adding one `WriterGroup` per group entry and one `DataSetWriter` per dataset. Each field is backed by a variable node in the server address space, updated every 100ms by a registered callback function that writes a random `Float32` value. Following

configuration, the publisher waits 10 seconds before enabling all PubSub components to allow the network configuration to stabilize, after which it begins transmitting at the configured interval.

On the publisher/server side every field node was connected to a callback function that gives the node a random `FLOAT32` value every 500 ms. The subscriber mirrors this topology in its own address space, reflecting the OPC UA PubSub information model. For each incoming dataset, a `DataSetReader` is configured with the matching `publisherId`, `writerGroupId`, and `dataSetWriterId`, which together uniquely identify the source and filter network messages accordingly. Received values are written directly into target variable nodes. The subscriber runs for the publisher duration plus a five-second grace period to ensure all in-flight messages are received before shutdown.

4.3.2 Client/Server using TCP

For the OPC UA server, only a standard configuration is needed. All values that should be observed need to be exposed as variables. To stay comparable to the UADP method, the server keeps the writer group $\rightarrow$ dataset $\rightarrow$ fields structure for the variables and a random callback is initialized for each. This callback updates the variable like the publishers every 100 ms and the datatype is also a `FLOAT32`.

The client then spawns one thread per server, plus a single embedded thread that acts as a local mirror and can be accessed as a TCP endpoint. Each publisher thread connects as an OPC UA client to the corresponding remote server. The client thread creates a single subscription at the configured `interval` and registers a monitored item for every field node in that publisher's subtree. When the server notifies a value change the callback immediately forwards the value to the corresponding node on the local embedded server using `UA_Server_writeValue`. This embedded server exposes all received values under a mirrored publisher $\rightarrow$ dataset $\rightarrow$ group $\rightarrow$ field hierarchy, making the aggregated data available to any downstream consumer. In total, $N + 1$ threads are spawned: one client thread per publisher and one thread driving the embedded server.

4.4 Parameter study

To experiment with the testbed a parameter study is created. The study is intended to demonstrate that the testbed is capable of producing clear and distinctive measurements across a range of configurations. An in-depth statistical analysis with multiple runs is considered out of scope for this thesis. Three key performance indicators (KPIs) are tracked throughout the exper-

iments: the CPU utilization of the application processes, the latency at the application layer, and the interface bandwidth inside the Docker containers. The parameters varied across the study are the number of publishers, the publish interval, the total number of data points transmitted per publish interval, and the distribution of those data points across writer groups and datasets. During the interval scaling study and the subsequent data point scaling and distribution studies, it became apparent that the debug logging overhead was too substantial and skewed the results significantly. Consequently, for all remaining studies a tighter distribution favouring a larger number of fields was adopted to minimize this effect. The network topology is additionally varied, configured as either a line, star or tree. The deployment environment is treated as a parameter as well, most notably in the final section where results from the SR-IOV testbed are compared against those obtained on the Raspberry Pi setup. Scenarios with multiple subscribers are not considered due to time and scope constraints. All measurements are therefore taken with a single subscriber.

4.4.1 Scaling Data points

The total number of data points transmitted per publish interval is determined by the number of fields per dataset writer, making field count the primary lever for scaling message payload size. While the publisher count and structural parameters remain fixed, increasing the field count directly increases the serialization overhead, message size, and consequently the bandwidth and CPU demands placed on the system. This study fixes 10 publishers, 5 writer groups and 5 dataset writers per publisher, scaling the number of fields from 1 up to 100, yielding a range of 25 to 4000 data points per publisher. This is done to simulate a populated and saturated deployment. The full parameter configuration is summarized in Table 1.

Table 1: Parameter configuration for the data point scaling study.

Parameter	Value
Publishers	10
Writer Groups	5
Dataset Writers	5
Topology	Tree
Deployment	SR-IOV
Publish Interval	1000 ms
Fields	1, 5, 10, 20, 40, 80, 160
Data Points (per Publisher)	25, 125, 250, 500, 1000, 2000, 4000

4.4.2 Scaling Intervals

The publish interval is one of the most fundamental configuration parameters in any publish-subscribe system, as it directly governs the rate at which data is produced and transmitted across the network. Varying the interval allows for an analysis of how the system behaves under different throughput demands. Shorter intervals increase the frequency of publish cycles, placing greater stress on CPU scheduling and network utilization, while longer intervals reduce load at the cost of data freshness.

The configuration used throughout this section fixes all structural parameters, with 500 data points per publisher chosen as a representative load that reflects a realistic mid-scale deployment without saturating the network under the shortest intervals tested. The full parameter configuration is summarized in Table 2.

Table 2: Parameter configuration for the interval scaling study.

Parameter	Value
Topology	Tree
Deployment	SR-IOV
Protocols	UDP, TCP
Publish Interval (ms)	500, 1000, 2000, 4000
Publishers	10
Writer Groups	5
Dataset Writers	2
Fields	50
Data Points (per Publisher)	500

4.4.3 Distribution Data points

Total data point count is fixed at 360 per publisher across all distribution configurations. The independent variable is the structural allocation of those points across writer groups, dataset writers, and fields within the UADP encoding. Writer groups and dataset writers impose distinct packetization and scheduling boundaries in OPC UA PubSub. Consequently, two configurations with identical total counts produce divergent network traffic patterns. The distribution axis spans from a single writer group encapsulating all 360 points in one dataset writer, to maximally fragmented structures in which numerous groups and writers each carry a minimal field count. Measurements are conducted over UDP with one publisher and one subscriber. The study configuration is given in Table 3; the per-distribution structural parameters are listed in Table 4.

Table 3: Parameter configuration for the data point distribution study.

Parameter	Value
Topology	Tree
Deployment	SRIOV
Protocol	UDP
Publish Interval	1000 ms
Publishers	1
Data Points (per Publisher)	360

Table 4: Data point distribution configurations, all yielding 360 data points per publisher.

Writer Groups	Dataset Writers	Fields
1	1	360
2	3	60
3	4	30
4	6	15
6	6	10
6	12	5
10	12	3

4.4.4 Scaling Publishers

The number of publishers represents a critical scaling dimension, as it directly determines the total load imposed on both the network infrastructure and the subscribing node. With up to 120 virtual functions allocatable on the SR-IOV testbed and one slot reserved for the subscriber, a theoretical maximum of 119 publishers can be deployed simultaneously. In practice, however, the underlying CPU exposes only 8 Performance and 16 Efficiency cores, meaning that at high publisher counts, containerized workloads begin to contend for physical cores, introducing interference that is architectural rather than network-bound.

To approach the hardware ceiling while remaining within a range where this effect is observable, the publisher count is scaled across eight steps cover-

ing the full range from a minimal configuration up to the VF limit. Both the UDP-based Publish-Subscribe and TCP-based Client-Server communication structures are evaluated, allowing a direct comparison of how each protocol handles increasing publisher load. This study is the first to adopt a tight distribution, consolidating data points into a single dataset writer with a high field count.

The per-publisher structure is therefore fixed at 2 writer groups and 1 dataset writer with 250 fields, yielding 500 data points per publisher. This configuration was chosen to limit the processing burden on the subscriber, as discussed in Section 4.5.3. The full parameter configuration is summarized in Table 5.

Table 5: Parameter configuration for the publisher scaling study.

Parameter	Value
Topology	Tree
Deployment	SR-IOV
Protocol	UDP, TCP
Publish Interval	1000 ms
Publishers	1, 10, 30, 50, 70, 90, 110, 119
Writer Groups	2
Dataset Writers	1
Fields	250
Data Points (per Publisher)	500

4.4.5 Different Topologies

To evaluate the influence of network topology on system behavior, three basic topologies are tested: star, line, and tree. Each is instantiated automatically within the virtual SR-IOV switch by a dedicated setup script, which constructs the required Linux bridges and veth interconnects for the selected topology at runtime. The TCP-based Client-Server structure is used exclusively in this study, as it avoids the multicast-related CPU overhead introduced by UADP, allowing latency differences attributable purely to topological path length and forwarding behavior to surface more clearly. The publisher count is fixed at 20 with 500 fields. The full parameter configuration is summarized in Table 6.

Table 6: Parameter configuration for topology comparison experiments. UDP publisher parameters are retained as discussed in Section 4.3.2 for consistency, though they have no bearing on the TCP configuration

Parameter	Value
Topologies	Star, Line, Tree
Deployment	SR-IOV
Protocol	TCP
Publish Interval	1000 ms
Publishers	20
Writer Groups	1
Dataset Writers	1
Fields	500
Data Points (per Publisher)	500

4.4.6 SR-IOV and Raspberry Pi Comparison

This experiment compares the SR-IOV virtualized deployment against a physical Raspberry Pi deployment across both UDP-based Publish-Subscribe and TCP-based Client-Server configurations. To address the primary research question of how latency and bandwidth on a virtual testbed compare to a physical deployment, the field count is scaled exponentially from 125 to 8000, yielding between 250 and 16 000 data points per publisher, with particular attention to CPU overhead under increasing data load.

With three publishers, the subscriber node receives up to 48 000 data points per interval under UDP and up to 24 000 under TCP. These upper bounds are chosen to approach and exceed the saturation thresholds reported by Burger et al. [59], who observe CPU saturation on a Raspberry Pi OPC UA system at approximately 20 000 data points for TCP and 40 000 for UDP. The experimental configuration is summarized in Table 7.

Where the Raspberry Pi encounters CPU constraints at higher data volumes, the SR-IOV deployment may offer a viable alternative that is not subject to the same limitations.

Table 7: Parameter configuration for the SR-IOV and Raspberry Pi comparison study.

Parameter	Value
Topology	Tree
Deployment	SR-IOV, Raspberry Pi
Protocol	UDP, TCP
Publish Interval	1000 ms
Publishers	3
Writer Groups	2
Dataset Writers	1
Fields	125, 250, 500, 1000, 2000, 4000, 8000 ^{UDP only}
Data Points (per Publisher)	250, 500, 1000, 2000, 4000, 8000, 16000 ^{UDP only}

4.5 Logging

The three KPIs of the study were logged via dedicated scripts executed on each node. For the cpu and bandwidth logs the `screen` utility [69] is chosen to run the logging processes in the background. After a set duration for the experiment this also allows to stop them with `screen -S cpu_monitor -X quit` and `screen -S bandwidth_monitor -X quit`.

4.5.1 CPU Usage

Two methods were considered for measuring CPU usage: querying the Docker daemon directly or reading from the cgroup filesystem. The Docker-based method yields a sampling granularity of approximately two seconds, imposed by the Docker daemon’s internal cgroup polling cycle. The cgroup-based method can provide sub-millisecond resolution but requires manual computation of CPU time deltas. Since CPU usage was not expected to exhibit sub-second variation relevant to the study, the Docker-based method was selected for its simplicity.

In Listing 1 the logging script is shown. At each iteration of the outer `while true` loop, the current wall-clock timestamp is captured via `TS=$(date +%T.%6N)`. Subsequently, `docker stats --no-stream` is invoked, which requests a single metrics snapshot from the Docker daemon for all running containers. The `-format` flag restricts the output to the container name and CPU utilization percentage, delimited by a comma. This output is piped

Listing 1: Script for logging CPU usage

```
screen -dmS cpu_monitor bash -c "
    while true; do
        TS=$(date +%T.%6N)
        sudo docker stats --no-stream --format '{{.Name}}
            {{.CPUPerc}}' | while IFS=, read NAME CPU;
        do
            echo "\"$TS, \"$NAME, \"$CPU\"
        done
    done | sudo tee -a logs/cpu.csv > /dev/null
"
```

into an inner `while` loop, where each line is split into the variables `NAME` and `CPU` using comma as the field separator (`IFS=,`). A CSV record is then written to stdout by prepending the captured timestamp, producing rows of the form `HH:MM:SS.ffffff,<container>,<cpu%>`. The full output is then piped to an appending tee to the `cpu.csv` file. The effective sampling interval is approximately two seconds, governed by the execution latency of `docker stats --no-stream`. It should be noted that `docker stats` reports CPU utilization relative to a single core rather than the total system capacity [70]. Consequently, values exceeding 100% are possible and indicate that the container’s workload spans multiple cores.

4.5.2 Bandwidth

Network bandwidth was logged using `sar` (System Activity Reporter), a utility from the `sysstat` package [71] that periodically samples and reports system resource metrics. The `-n DEV` flag instructs `sar` to report per-interface network statistics at a specified interval. Each sample produces a row containing the timestamp, interface name, received and transmitted packet rates (`rxpck/s`, `txpck/s`), received and transmitted throughput in kilobytes per second (`rxkB/s`, `txkB/s`), compressed packet rates (`rxcmp/s`, `txcmp/s`), multicast packet rate (`rxmcst/s`), and interface utilization percentage (`%ifutil`).

On the Raspberry Pi nodes, `sar` was executed directly on the `eth0` interface. On the SR-IOV host node, `sar` was executed within the respective namespace using `sudo ip netns exec`, targeting the corresponding VLAN interface. Using it on the VF-interface or the PF-interface wouldn’t have worked as their traffic is offloaded to hardware and not seen by the kernel.

As seen in Listing 2, it was necessary to prepend `sudo env LC_ALL=C` to the command in both cases. Without explicitly setting the locale to `C`, `sar`

Listing 2: Script for logging bandwidth for an interface

```
screen -dmS bandwidth_monitor bash -c "  
sudo env LC_ALL=C sar -n DEV 1 | grep -E 'IFACE|${  
    INTERFACE}' | sudo tee logs/bandwidth_${INTERFACE}_log  
    > /dev/null  
"
```

produces output formatted according to the system locale, which introduced decimal separators or AM/PM timestamp formats inconsistent with the expected format, causing parsing errors during post-processing. Output was filtered with `grep` to retain only the header line and the relevant interface, and written to a log file via `tee`.

4.5.3 Latency

Latency was measured at the application level to reflect the end-to-end delay as experienced by a user of each system. Since the built-in logger only provides millisecond granularity, a custom logger was implemented that captures only the relevant log lines from each container into `<container_name>_log`.

TCP Extracting timestamps on the client side required first isolating each server's TCP connection via the log line:

```
[<timestamp>] info/network TCP <connection_id> |  
Opening a connection to <server_address> on port 4840
```

This mapped each server address to an internal `connection_id`, which subsequently appeared whenever a message was processed:

```
[<timestamp>] debug/channel TCP <connection_id> | SC 1 |  
Process MSG message with RequestId <request_id>
```

Because the server responds using the `request_id` it receives, but does not indicate the originating request type, responses from requests such as `CreateMonitoredItemsRequest` or `GetEndpointsRequest` are structurally indistinguishable in the logs. To resolve this ambiguity, outgoing request lines of the form:

```
[<timestamp>] debug/channel TCP <connection_id> | SC 1 |  
Sending request with RequestId <request_id> of type  
<request_type>
```

were also captured, allowing each `request_id` to be mapped to its `request_type` and thereby isolating latency measurements to data transfer messages only.

On the server side, each outgoing publish response is logged as:

```
[<timestamp>] debug/session TCP <connection_id> | SC 1 |
Session "ns=1;g=<session_id>" | Subscription <subscription_id>
| Sending out a publish response with <data_points>
notifications
```

Since the experiment involved a single subscriber and no packet loss simulation, this line was sufficient to pair each publish request with its corresponding response. In summary, the TCP filters applied were `Opening a connection` and `RequestId` on the client side, and `publish response` on the server side.

UDP Log filtering for UDP was more straightforward. Each time a writer group publishes, the following line is emitted:

```
[<timestamp>] debug/pubsub PubSubConnection ns=1;i=50039 |
WriterGroup ns=1;i=<writerGroupNode_id> | Publish Callback
```

On the receiving end, every dataset reader logs its arrival independently:

```
[<timestamp>] debug/pubsub PubSubConnection ns=1;i=50039 |
ReaderGroup i=0 | DataSetReader ns=1;i=<dataSetReaderNode_id>
| Received a network message
```

Because a single writer group bundles multiple datasets into one transmission, a single publish event produces multiple receive-side log entries. To obtain a single representative receive timestamp per message, only the earliest (minimum) timestamp across all dataset readers is used. The mapping between `writerGroupNode_id` and its associated `dataSetReader_ids` is written out by the application at startup and consumed by the analysis script.

Debug mode overhead Capturing the required log lines necessitates running the application in debug mode, which introduces non-trivial CPU overhead. This is particularly pronounced in the UADP configuration, where the overhead scales with the number of publishers and subscription layers. Specifically, the open62541 stack emits a log line at each level of the receive hierarchy for every incoming pack, when the ID doesn't match. For a topology of P publishers, G writer groups, and D datasets, each transmitted packet generates therefore $P - 1$ log lines at the `PubSubConnection`

level, $G - 1$ at the `ReaderGroup` level, and D at the `DataSetReader` level, as each packet triggers four ID-mismatch rejections and one successful receive acknowledgement before being processed. This yields $P + G + D - 2$ log lines per packet. Since for each publisher each writer group transmits its own packet, this cost is incurred G times per publish interval, resulting in $(P + G + D - 2) \cdot GP$ log lines per interval per node. In the experimental configuration of $P = G = D = 5$, this amounts to 325 log lines per packet on the subscriber node, imposing a measurable burden on the CPU that must be accounted for when interpreting the CPU utilization measurements.

For the TCP configuration logging overhead consists of noise around each notification as each step of sending the notification is logged, so for more notifications as in case of shorter intervals or bigger field sizes the overhead in debug mode should noticeably increase.

5 Evaluation

5.1 Scaling Data Points

This study simulates a populated deployment by fixing 10 publishers, 5 writer groups, and 5 dataset writers, while scaling the field count from 1 to 160, yielding between 25 and 4000 data points per publisher. With publisher count, writer group count, and dataset writer count all held constant, the debug mode overhead described in Section 4.5.3 contributes a fixed additive cost independent of field count.

CPU

On the subscriber side, shown in Figure 11, TCP and UDP exhibit qualitatively distinct behavior. For UDP, debug mode utilization remains approximately constant across all field counts, fluctuating between 26.7% and 32.9% with no discernible trend, while the no-debug series increases monotonically from 2.07% at 25 data points to 6.40% at 4000. The large stable gap between the two series confirms that logging overhead dominates subscriber CPU under UDP and is insensitive to field count, as expected from the fixed structural parameters.

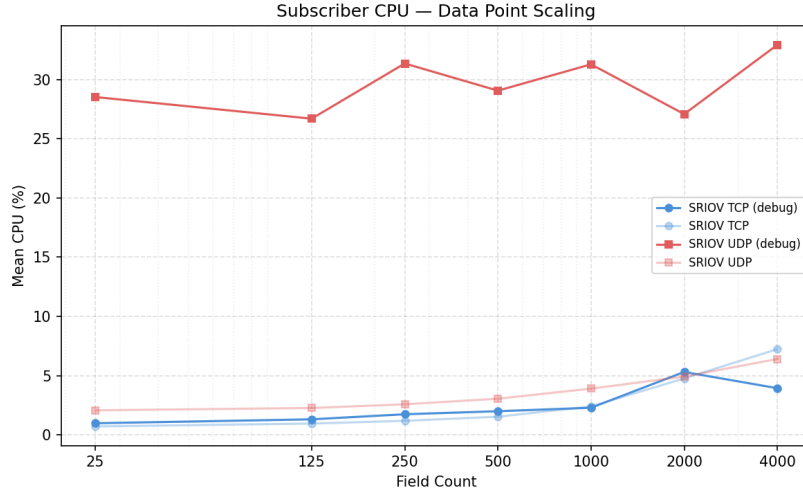


Figure 11: Subscriber CPU utilization across data point counts for TCP and UDP, with and without debug logging. Field Count is logarithmic to space out results.

For TCP, both series increase with field count. The debug series exceeds the no-debug series throughout most of the range, with the gap narrowing

at higher counts. At 4000 data points the ordering inverts, with no-debug at 7.24% and debug at 3.94%; this is attributed to measurement variance. That no-debug TCP publisher utilization exceeds no-debug UDP at equivalent data point counts is consistent with the per-notification processing overhead inherent to the session-based TCP model.

On the publisher side, shown in Figure 12, the UDP debug and no-debug series are nearly coincident, rising from below 0.1% to approximately 1.25% at 4000 data points. The small separation is consistent with the fixed log line volume and low absolute UDP publisher load.

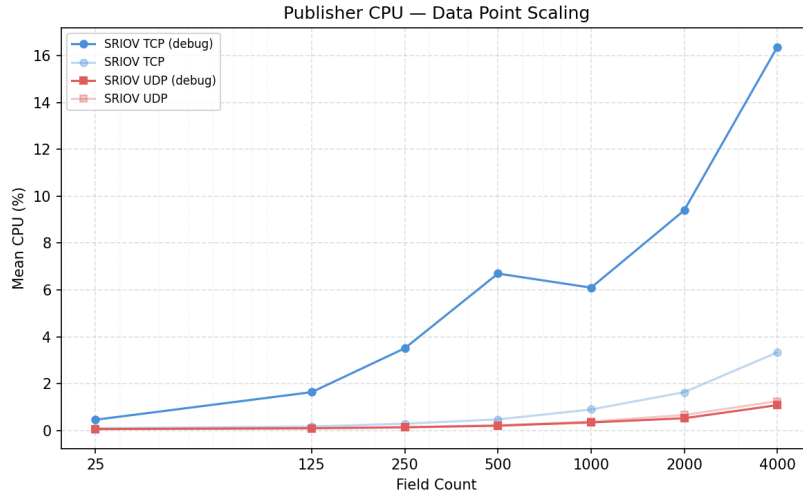


Figure 12: Publisher CPU utilization across data point counts for TCP and UDP, with and without debug logging. Field Count is logarithmic.

For TCP, the debug series scales from 0.46% at 25 data points to 16.37% at 4000, while no-debug rises from 0.10% to 3.34%. The growing ratio, reaching approximately $4.9\times$ at 4000 data points, indicates that publisher logging cost under TCP scales with notification volume rather than remaining fixed. The drop in debug utilization between 500 and 1000 data points is inconsistent with the surrounding trend and is attributed to a measurement artefact.

Bandwidth

Publisher and subscriber bandwidth scale linearly with data point count across both protocols, as shown in Figures 13 and 14. TCP publisher transmit grows from 0.87 kB/s at 25 fields to 102.7 kB/s at 4000, with the subscriber receiving the corresponding aggregate of three publishers. UDP transmit

follows the same linear trend at a lower absolute level, consistent with the more compact UADP encoding relative to the TCP session overhead.

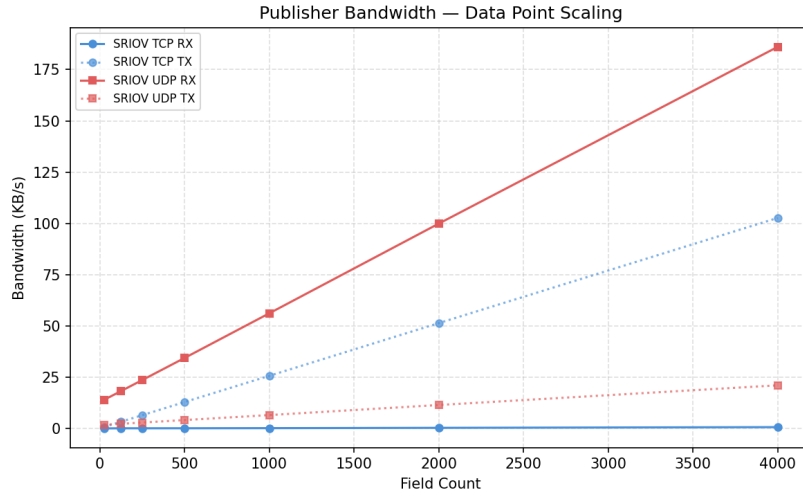


Figure 13: Average per-publisher transmit and receive bandwidth for TCP and UDP across increasing data point counts.

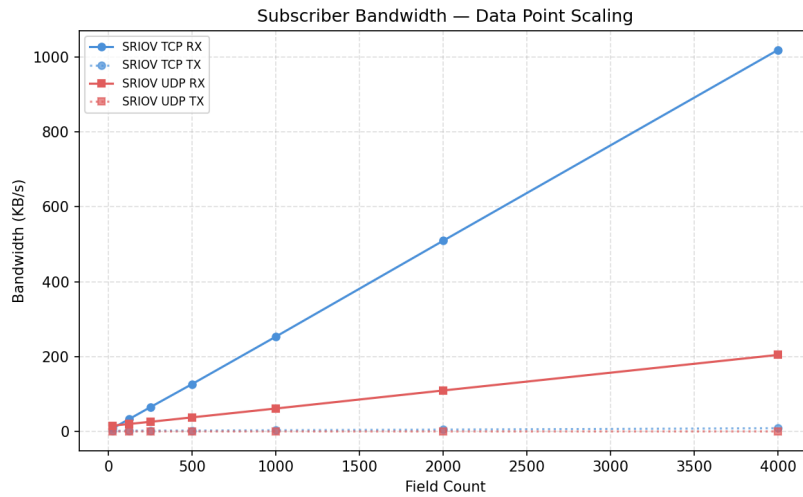


Figure 14: Subscriber receive and transmit bandwidth for TCP and UDP across increasing data point counts.

A notable asymmetry appears on the UDP publisher receive side, where incoming bandwidth grows proportionally alongside transmit, reaching 186 kB/s

at 4000 fields. This reflects the absence of multicast group filtering in the virtual switch, leading to multicast flooding each publisher.

Latency

TCP median latency increases from 0.36 ms at 25 data points to 1.50 ms at 4,000, consistent with serialization overhead scaling with payload size, as shown in Figure 15. The anomalous drop at 500 fields is attributed to measurement variance.

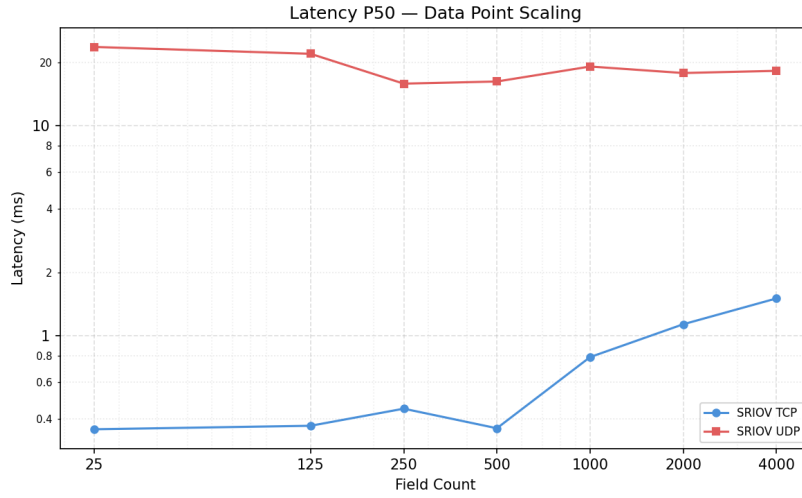


Figure 15: Median latency across data point counts for TCP and UDP in log-log scale

UDP median latency fluctuates between 15 and 24 ms with no consistent trend, indicating that payload size does not significantly influence UDP latency within this range. The elevated baseline is attributed to logging overhead on the subscriber side, where timestamps are captured after their turn in the logging queue rather than at message arrival.

The Cumulative Distribution Function(CDF) in Figure 16 shows the TCP distribution latencies concentrated below 10 ms with a short tail, while the UDP distribution spreads across a range exceeding 100 ms. The gap between the two protocols exceeds any variation attributable to data point count and suggest that TCP is for the current configuration the preferable protocol.

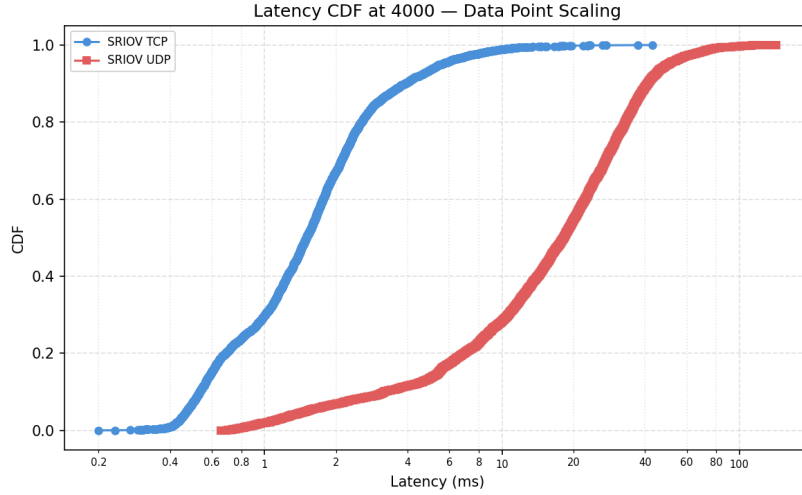


Figure 16: CDF of latency for TCP and UDP at 4,000 data points. Latency in log scale

5.2 Scaling Intervals

The following results show whether a measurable impact on latency bandwidth and cpu usage can be achieved by scaling up the interval size of the publishers.

CPU

On the publisher side, shown in Figure 17, TCP and UDP no-debug utilization remains flat across all intervals at approximately 0.45 % and 0.18 % respectively. The TCP debug series shows a slight upward drift from 5.2 % to 7.6 % with increasing interval, however no causal mechanism consistent with the system's operation could be identified. The variation is therefore treated as measurement noise.

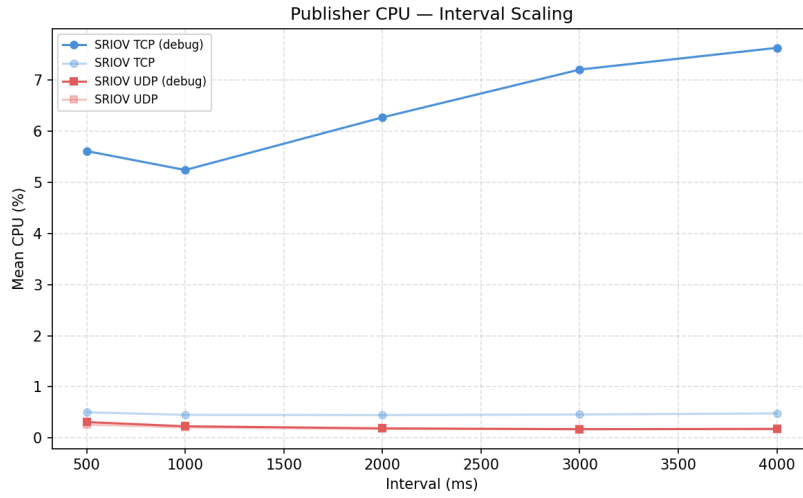


Figure 17: Publisher CPU utilization across publish intervals for TCP and UDP.

On the subscriber side, shown in Figure 18, TCP utilization decreases with longer intervals from 3.1 % to 1.1 % in debug mode and from 2.6 % to 0.94 % without, consistent with fewer publish responses arriving per unit time. UDP no-debug follows the same pattern, falling from 2.2 % to 0.34 %. UDP debug utilization drops sharply from 26.4 % at 500 ms to 4.0 % at 4,000 ms, reflecting the fixed per-packet logging overhead scaling directly with message arrival rate.

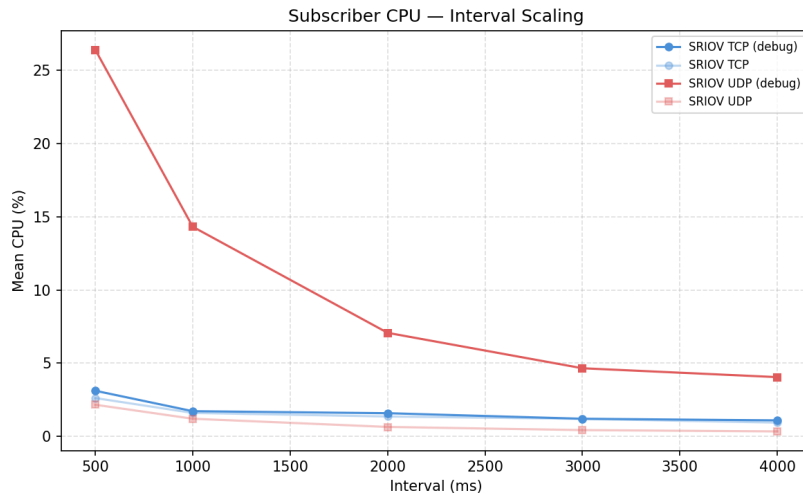


Figure 18: Subscriber CPU utilization across publish intervals for TCP and UDP.

Bandwidth

Publisher and subscriber bandwidth, shown in Figures 19 and 20, halves with each doubling of the publish interval for both protocols, consistent with a fixed message size transmitted at half the rate. Both figures otherwise reflect the protocol behavior described in Section 5.1.

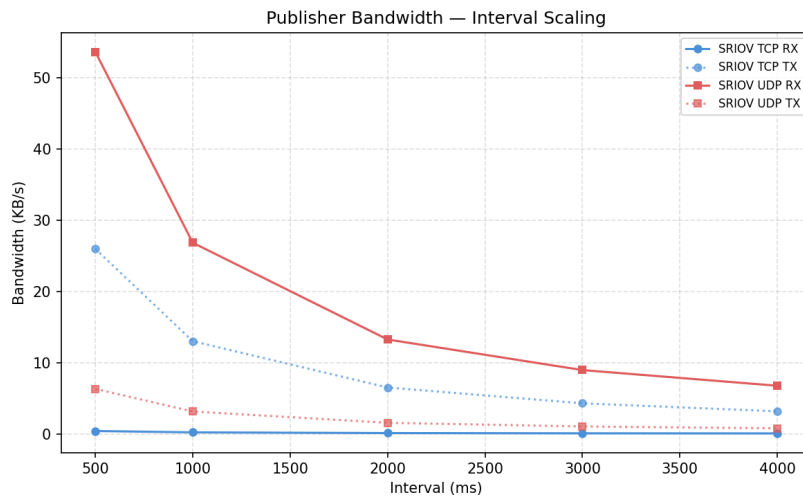


Figure 19: Publisher bandwidth across publish intervals for TCP and UDP.

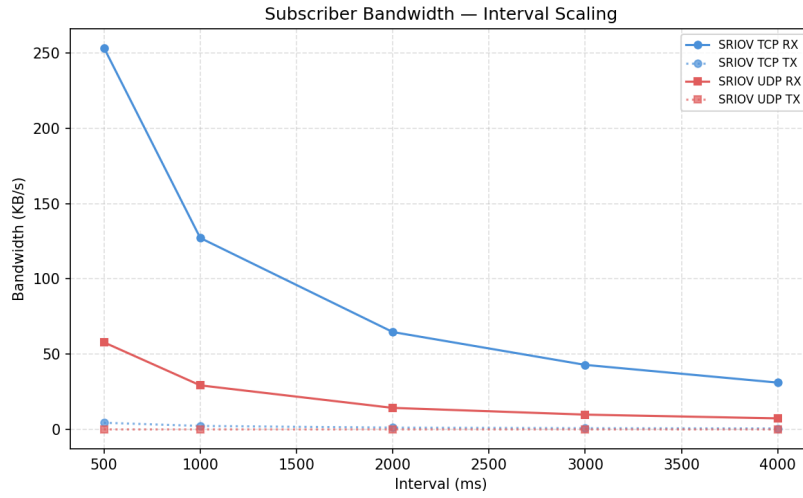


Figure 20: Subscriber bandwidth across publish intervals for TCP and UDP.

Latency

Median latency, shown in Figure 21, remains stable across all intervals for both protocols. TCP p50 fluctuates narrowly between 0.46 ms and 0.53 ms, and UDP p50 between 6.25 ms and 7.5 ms, indicating that the publish interval has no meaningful effect on median latency. This is consistent with the median being dominated by logging overhead rather than network timing.

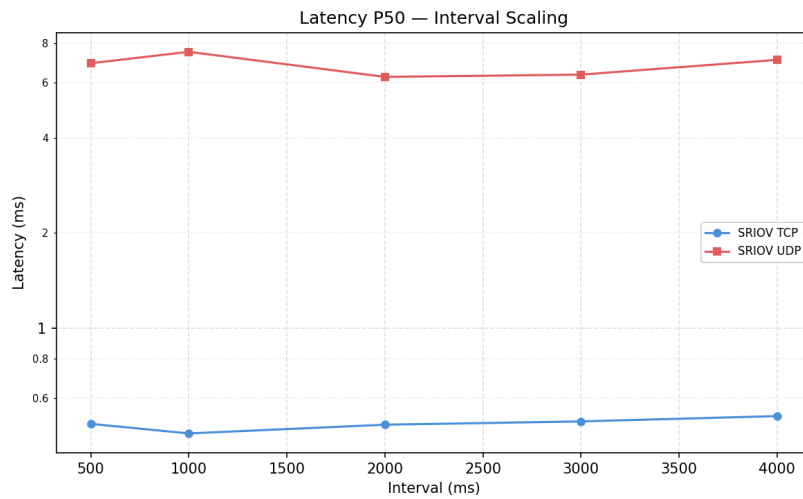


Figure 21: Median latency across publish intervals for TCP and UDP (logarithmic scale).

The 95th percentile, shown in Figure 22, reveals a more informative picture. TCP p95 remains below 2 ms across most intervals, with the elevated values at 2,000 ms and 3,000 ms attributed to measurement noise. UDP p95 at 500 ms and 1,000 ms intervals reaches approximately 1,017 ms and 1,009 ms respectively, matching one full publish cycle, indicating that a substantial fraction of messages are delayed by an entire interval before being logged. Beyond 1,000 ms this effect disappears, with UDP p95 dropping to the 14–16 ms range, showing a way to reduce overhead for wide distributions.

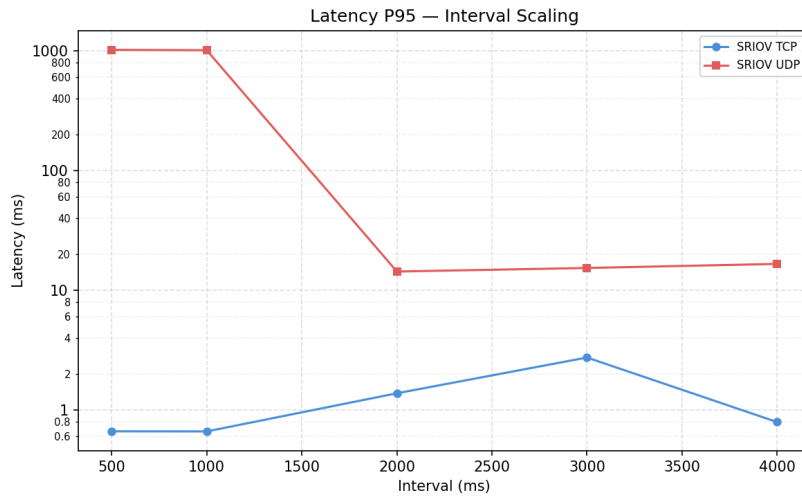


Figure 22: 95th percentile latency across publish intervals for TCP and UDP (logarithmic scale).

5.3 Distribution

The following results examine whether the structural organization of data points within a publisher has any measurable effect on the system behavior, or whether only the total payload size matters.

Latency

At the least fragmented configuration (1 writer group, 1 dataset writer, 360 fields), the median latency sits at 1,054 μ s with a p95 of just 1,405 μ s, indicating a tight and predictable distribution. As fragmentation increases, latency degrades rapidly. The 4-6-15 configuration already shows a median of 3,240 μ s, and the most fragmented configuration tested (6-12-5) reaches a median of 16,103 μ s with a p95 of 33,475 μ s, which is more than an order of magnitude worse than the least fragmented case. This behavior is well

illustrated by the CDF in Figure 25. The least fragmented configuration produces a steep, narrow curve concentrated around 1 ms, while the most fragmented configuration spreads across a range exceeding 20 ms with a long tail. The gap between p50 and p95 also widens significantly with fragmentation, pointing to increasing jitter rather than a uniform shift. This suggests that the overhead stems from the increased per-message processing cost incurred when the subscriber must handle many small datasets from multiple writer groups within each publish interval.

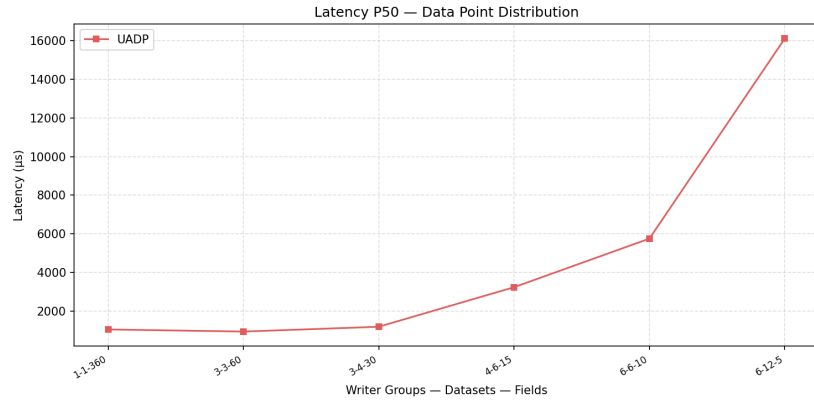


Figure 23: Median (p50) latency across distribution configurations, showing a clear trend from least fragmented to most fragmented.

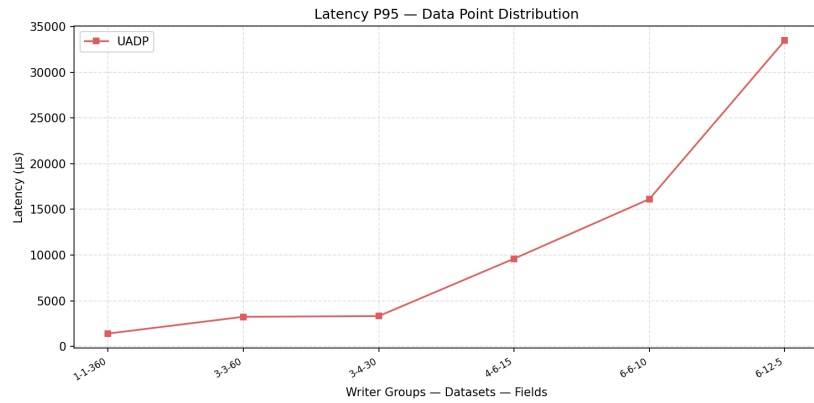


Figure 24: 95th percentile (p95) latency across distribution configurations, showing the same trend as the p50 latency.

This behavior is well illustrated by the CDF in Figure 25. The least fragmented configuration produces a steep, narrow curve that is concentrated between 1 ms and 2 ms, while the most fragmented configuration spreads across

a range exceeding 20 ms with a long tail. The gap between p50 and p95 also widens significantly with fragmentation, pointing to increasing jitter rather than a uniform shift. This suggests that the overhead does not stem solely from larger message sizes but from the increased per-message processing cost incurred when the subscriber must handle many small datasets from multiple writer groups within each publish interval as described in Section 4.5.3

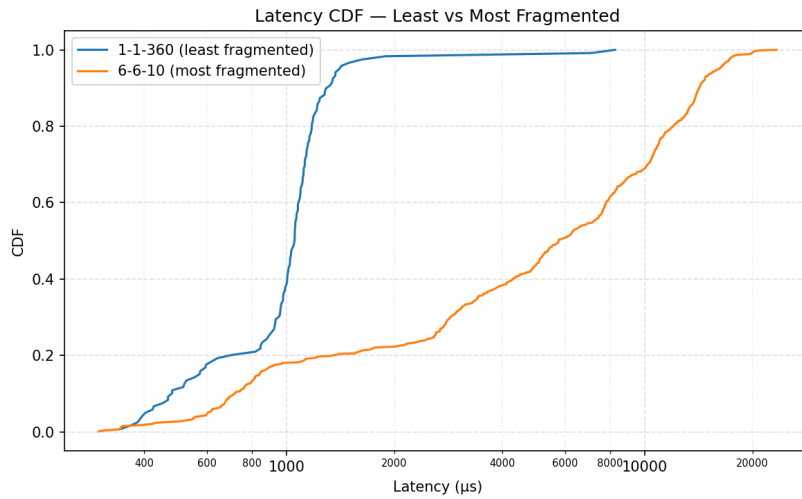


Figure 25: Cumulative Distribution Function of latency across distribution configurations.

Bandwidth

The bandwidth results follow a similar pattern to latency, with more fragmented configurations consuming noticeably more network capacity. On the publisher side, transmit bandwidth increases from 1.83 KB/s for the least fragmented configuration (1-1-360) up to 6.73 KB/s for the most fragmented (6-12-5), despite the total number of data points remaining constant at 360 across all configurations. A possible reason for this development is that each additional writer group and dataset writer introduces its own network message with associated headers and metadata, most notably the writer group identifiers and network masks required by the pub/sub addressing scheme. More datasets therefore means more messages per publish interval, each carrying this fixed overhead regardless of their payload size.

The subscriber side mirrors this trend closely, with receive bandwidth rising from 1.77 KB/s to 5.78 KB/s across the same range of configurations, consistent with the increased number of messages arriving per interval. The

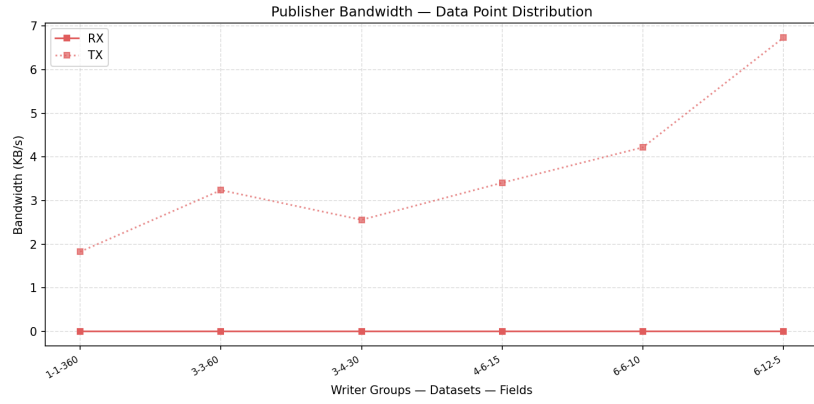


Figure 26: Publisher transmit and receive bandwidth across distribution configurations.

receive bandwidth on the publisher side and the transmit bandwidth on the subscriber side remain negligibly small throughout, as expected for a unidirectional pub/sub flow.

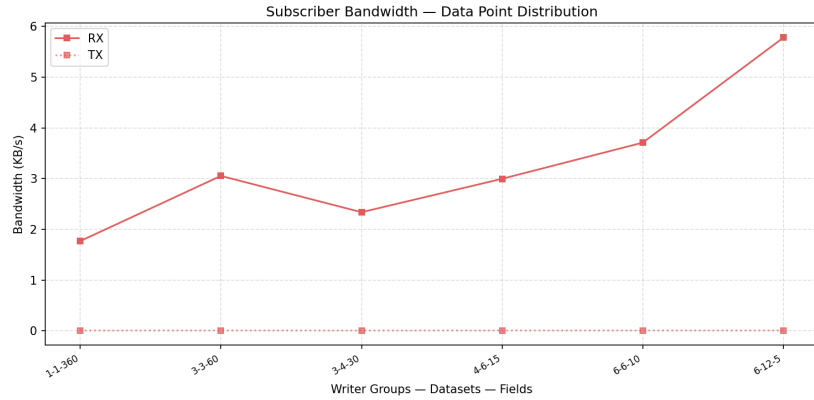


Figure 27: Subscriber receive and transmit bandwidth across distribution configurations.

If the overhead is indeed driven purely by fixed-size framing, the relative bandwidth difference between configurations would diminish at larger payload sizes, as the per-message header cost becomes a smaller fraction of the total. Verifying this remains outside the scope of this study.

CPU

The CPU results reveal a striking asymmetry between publisher and subscriber. On the publisher side, CPU utilization remains largely stable across all configurations, ranging from 0.63 % to 1.01 %, with no clear trend attributable to fragmentation. The subscriber tells a fundamentally different story. Without debug logging, subscriber CPU remains low throughout, rising modestly from 0.06 % to 0.15 % across all configurations. With debug logging enabled, however, subscriber CPU scales aggressively with fragmentation, increasing from 0.07 % at the least fragmented configuration to 2.75 % at the most fragmented.

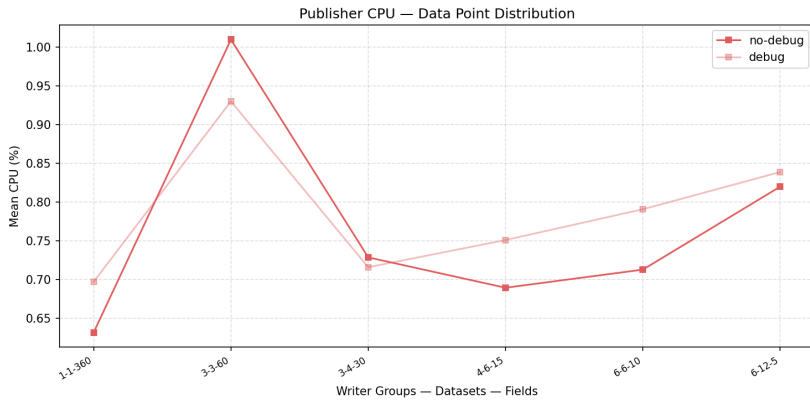


Figure 28: Publisher CPU utilization across distribution configurations.

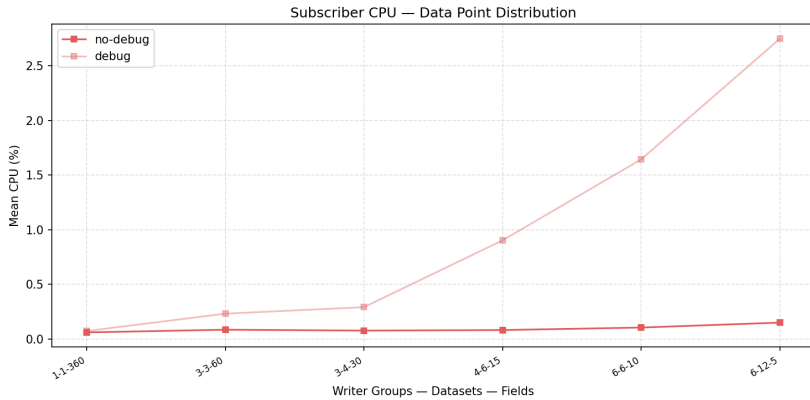


Figure 29: Subscriber CPU utilization across distribution configurations, with and without debug logging. A sharp increase of cpu overhead can be observed as to the polynomial nature of the logging.

This is a direct consequence of the $(P + G + D - 2) \cdot P \cdot G$ log line scaling described in Section 4.5.3. As the number of writer groups and dataset writers increases across configurations, the per-packet log output grows accordingly, driving up subscriber CPU independently of any real processing load. These results confirm that debug mode overhead is not a fixed cost but one that compounds with distribution complexity, and directly motivate the adoption of a tight distribution in all subsequent studies to prevent logging artefacts from obscuring the effects of the parameters under investigation.

5.4 Publisher Scaling

This study uses a tight distribution with all data points consolidated into a single dataset writer with a high field count, as established in Section 5.3. No-Debug logging is disabled for this and all subsequent studies, as the Debug versus No-Debug comparison established in prior sections is no longer required.

CPU

On the publisher side, shown in Figure 30, TCP cumulative utilization scales from 5.5 % at one publisher to a peak of 396 % at 90, approaching what seems to be the physical ceiling. Beyond this point utilization declines to 386 % at 110 publishers and 276 % at 119, indicating that the available CPU budget is increasingly contested between publisher and subscriber containers. UDP cumulative publisher utilization scales more gradually from 0.22 % to a plateau of approximately 15–16 % beyond 70 publishers, leaving substantial headroom throughout.

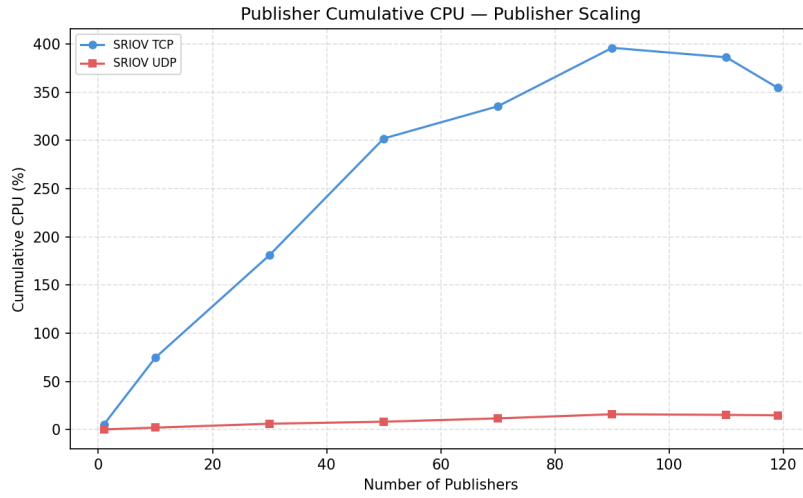


Figure 30: Cumulative publisher CPU utilization across publisher counts for TCP and UDP.

On the subscriber side, shown in Figure 31, TCP utilization increases from 0.23% at one publisher to 17.4% at 70, then stabilizes in the 15–18% range through to 119 publishers. This plateau is consistent with the subscriber being limited by the rate at which publishers can deliver data as the aggregate CPU load approaches the physical ceiling. UDP subscriber utilization increases continuously from 0.10% to 45.7% at 119 publishers, reflecting a steady increase in incoming message rate without the contention effects observed under TCP.

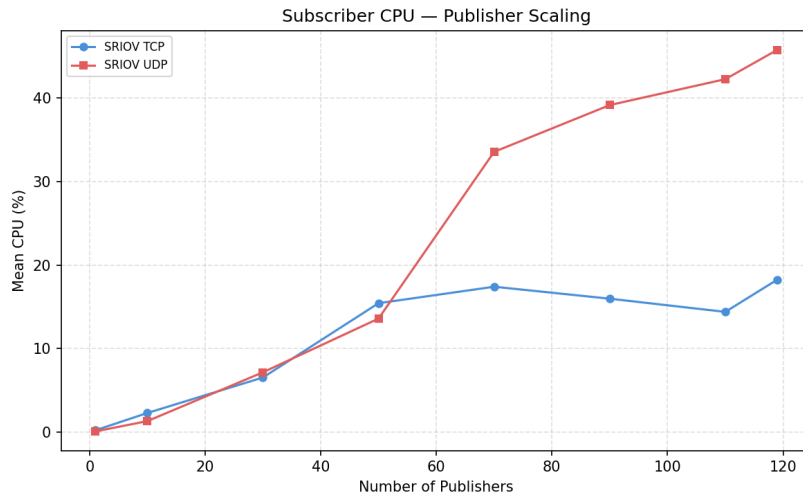


Figure 31: Subscriber CPU utilization across publisher counts for TCP and UDP.

Bandwidth

Per-publisher transmit bandwidth remains constant across all publisher counts for both protocols, as shown in Figure 32, stabilizing at approximately 13.0 kB/s for TCP and 2.59 kB/s for UDP. This holds across the full range from 1 to 119 concurrent publishers, indicating that resource contention at higher publisher counts does not measurably reduce the transmission rate of individual publishers.

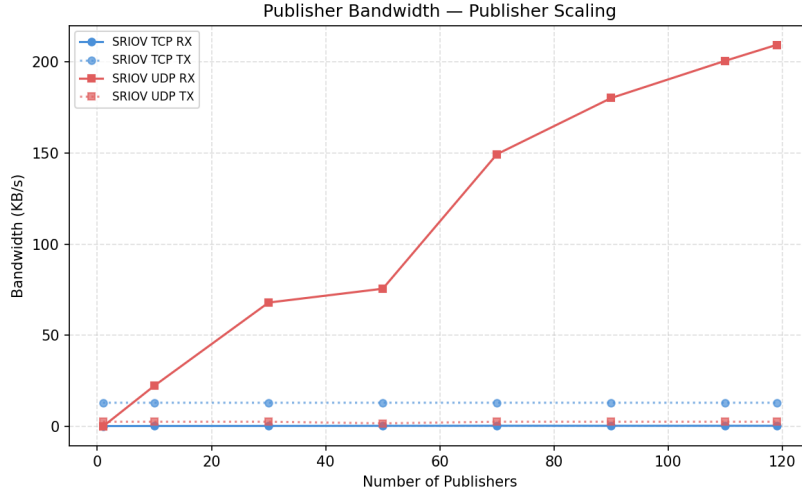


Figure 32: Average per-publisher transmit and receive bandwidth across increasing publisher counts for TCP and UDP. SR-IOV UDP receive bandwidth is omitted for clarity, as the values reflect multicast flooding within the virtual switch rather than application traffic, as discussed in Section 5.1.

Subscriber receive bandwidth, shown in Figure 33, scales proportionally with publisher count in both protocols, reaching approximately 1213 kB/s for TCP and 190 kB/s for UDP at 119 publishers, consistent with the fixed per-publisher transmission rate observed above. Subscriber transmit bandwidth remains negligible under UDP throughout the evaluated range, as the brokerless transport generates no acknowledgement traffic. Under TCP, subscriber transmit bandwidth increases gradually with publisher count, which is consistent with the per-connection keepalive and acknowledgement overhead of each additional persistent TCP session maintained by the subscriber.

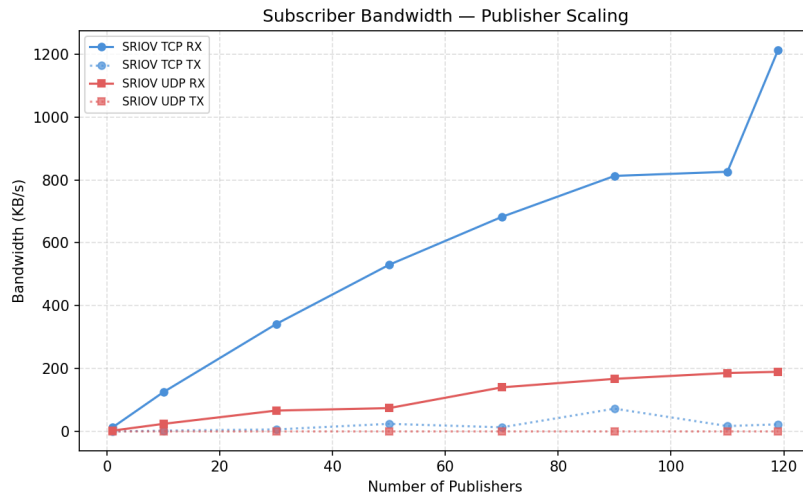


Figure 33: Subscriber receive and transmit bandwidth across increasing publisher counts for TCP and UDP.

Latency

Figure 34 shows median latency across increasing publisher counts for both protocols. TCP latency decreases monotonically from 0.59 ms at 1 publisher to 0.26 ms at 119, which may reflect more efficient notification batching under higher server load, though a definitive interpretation would require further investigation.

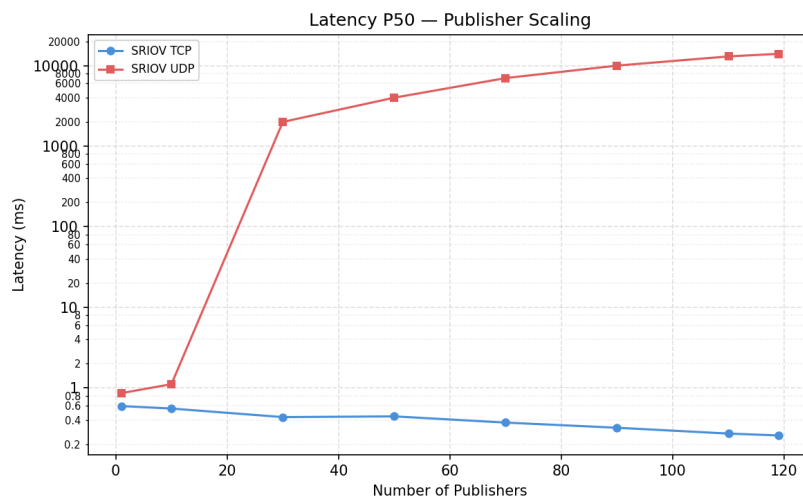


Figure 34: Median (p50) latency across increasing publisher counts for TCP and UDP.

UDP latency remains below 2 ms up to 10 publishers before increasing approximately linearly at roughly 100 ms per additional publisher, reaching 14 s at 119 publishers. This growth pattern is consistent with the debug logging overhead described in Section 4.5.3, where the number of log lines emitted per packet scales polynomial with publisher and writer group count. At higher publisher counts this overhead is sufficient to delay packet processing on the subscriber to a degree that dominates the measured end-to-end latency. These results should therefore not be interpreted as representative of the network-level latency of the UDP transport under production conditions.

5.5 Different Topologies

This section evaluates the influence of network topology on observed latency and bandwidth. CPU utilization is not reported here, as the topology variable affects only the forwarding path on the switch node and does not meaningfully alter the processing load on publisher or subscriber containers.

The latency results in Figure 35 follow the expected structural ordering. The star topology yields the lowest median at 528 μ s, as every publisher shares a single bridge with the subscriber. The tree records 541 μ s, reflecting the additional aggregation hop traversed by traffic originating at leaf bridges. The line topology produces the highest median at 571 μ s, explained by the fact that a packet from the farthest publisher must cross all 20 intermediate bridges sequentially before reaching the subscriber.

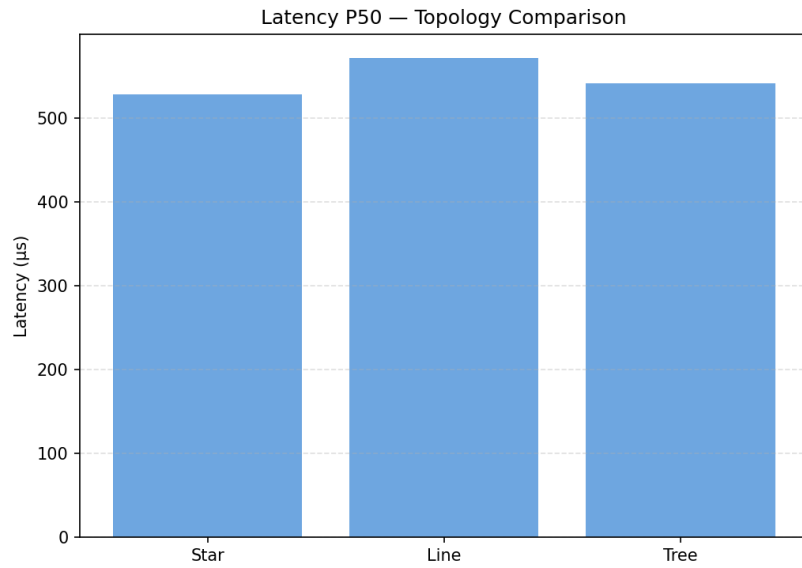


Figure 35: Median (p50) latency across the three evaluated topologies for the TCP Client-Server configuration with 20 publishers. Expected differences between the three topologies can be observed.

Bandwidth, shown in Figures 36 and 37, is unaffected by topology. Per-publisher transmit bandwidth remains stable at approximately 26 kB/s across all three configurations, and subscriber receive bandwidth at approximately 475 kB/s, confirming that topology influences path latency only and not aggregate throughput.

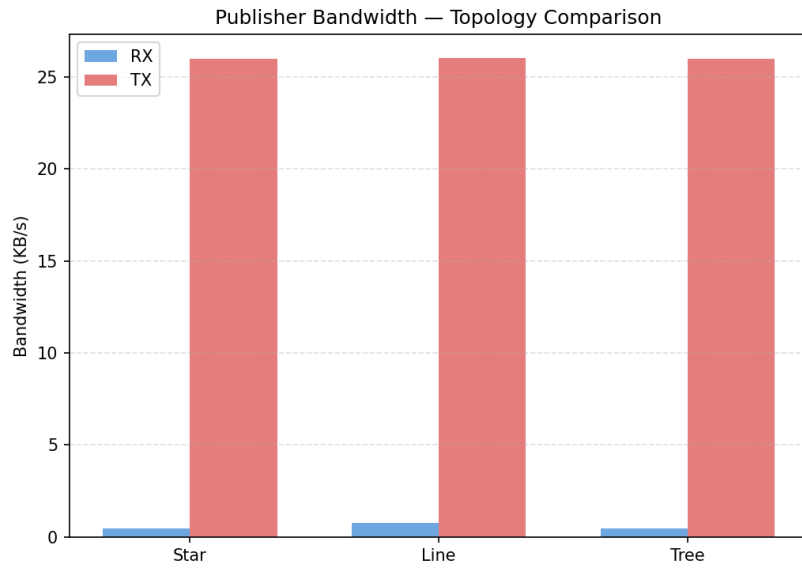


Figure 36: Average per-publisher transmit and receive bandwidth across the three evaluated topologies.

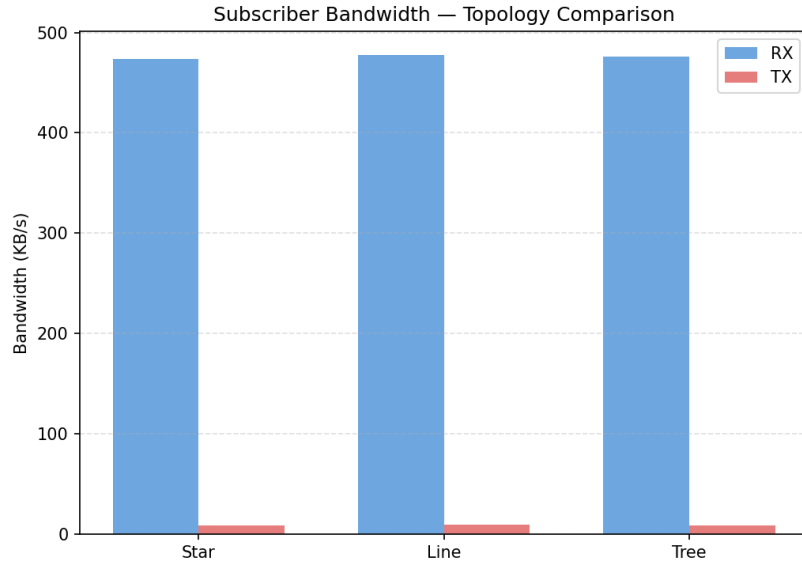


Figure 37: Subscriber receive and transmit bandwidth across the three evaluated topologies.

The absolute latency differences are small, spanning $43\ \mu\text{s}$ across all three topologies, which is consistent with the additional forwarding steps being

implemented entirely in software without physical propagation delays. Nevertheless, the directional ordering is preserved and structurally consistent, suggesting that an SR-IOV testbed can reproduce topology-dependent latency behavior qualitatively. Further investigation across larger and more complex topologies, using network emulation tools to increase delays, would be required to characterize the effect at scale.

5.6 SRIOV and PI comparison

Comparing the effects between the SRIOV and the PI testbed is the most important research question. In the evaluation it is interesting too see how much latency differs, what effects the added cpu have and if previously recorded data [59] can be validated.

CPU evaluation

TCP. On the publisher side (Figure 38), PI utilization scales approximately linearly up to 2000 datapoints, reaching 84.7 %, before appearing to saturate at 4000 datapoints (103.2 %). The marginal decrease to 101.1 % at 8000 datapoints is consistent with a CPU-bound regime in which the server is no longer able to sustain the nominal publication rate, though further measurements would be required to confirm this interpretation. A plausible downstream consequence of such saturation would be a reduction in effective message throughput, which may in turn manifest on the subscriber side. The subscriber data visualized in Figure 39 lends support to this hypothesis. The PI subscriber utilization peaks at 32.2 % at 4000 datapoints before falling to 14.4 % at 8000, a pattern more consistent with a reduction in incoming message rate than with any decline in the subscriber’s own processing capacity. SR-IOV remains well within its computational budget throughout, reaching 47.1 % on the publisher and 2.7 % on the subscriber at 8000 datapoints, and shows no indication of approaching saturation across the evaluated range.

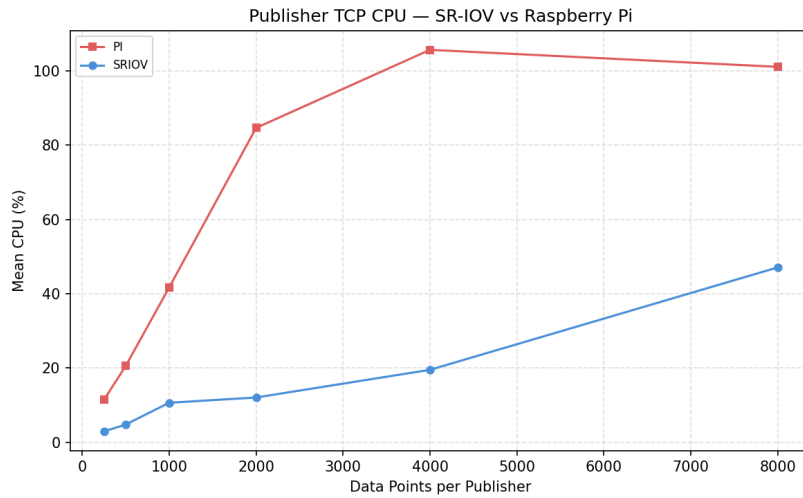


Figure 38: CPU comparison between SR-IOV and PI on the TCP publisher. At 4000 Data Points a saturation effect sets in. CPU values exceed 100% as `docker stats` scales by core count (see Section 4.5.1).

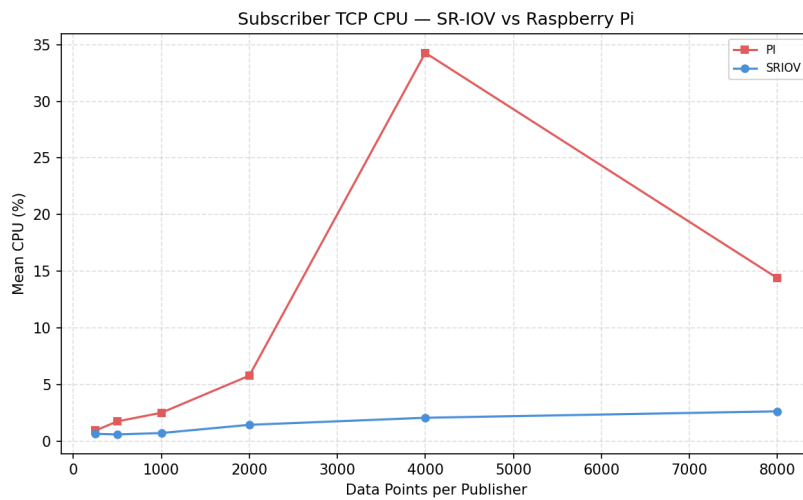


Figure 39: CPU comparison between SR-IOV and PI on the TCP subscriber. A steep drop of correlated to saturation on the publisher side can be seen at 8000 Data Points.

UDP. The UDP workload (Figures 40 and 41) presents a qualitatively different profile. PI publisher utilization remains below 3% up to 2000 data-points before rising to 66.1% at 16000, suggesting that the PI configuration

may be approaching its processing ceiling at the upper end of the evaluated range, though it does not appear to cross it within the measured configurations. The subscriber side exhibits no analogous pressure, PI subscriber utilization reaches only 18.5 % and SR-IOV 2.7 % at 16 000 datapoints, with both series following smooth, monotonically increasing trends.

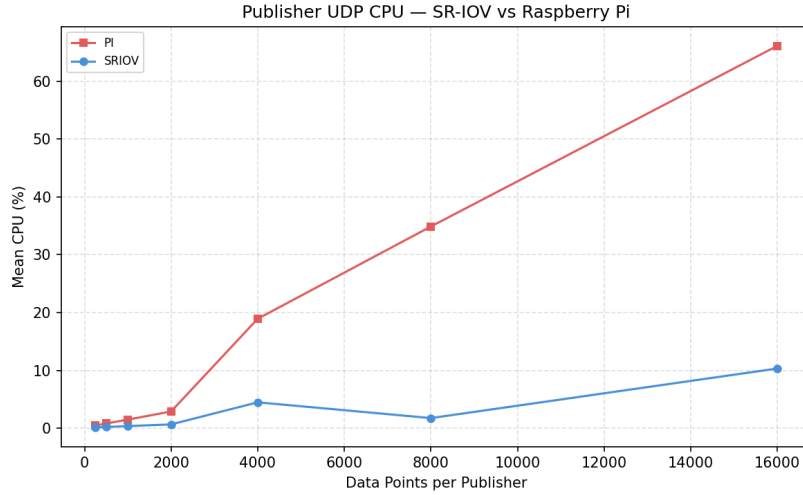


Figure 40: CPU comparison between SR-IOV and PI on the UDP publisher. SR-IOV utilization remains consistently low across all configurations, while the Raspberry Pi exhibits a pronounced nonlinear increase beyond 2000 data points, reaching 66% at 16000 data points. The anomalous SR-IOV measurement at 8000 data points is likely attributable to measurement variance and should be interpreted with caution.

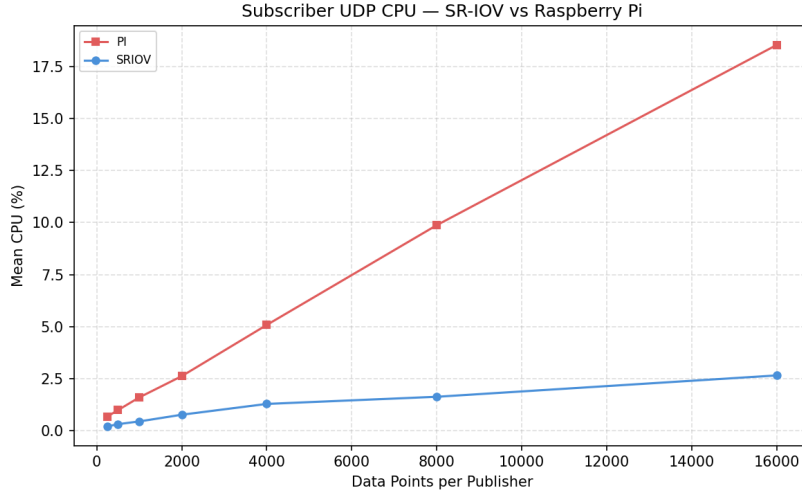


Figure 41: CPU comparison between SR-IOV and PI on the UDP subscriber. A near linear increase of CPU utilization can be observed.

Publisher as bottleneck. Across both protocols, the available evidence suggests that the publisher represents the more likely performance-limiting component within the evaluated configuration space. The subscriber, while subject to a capacity ceiling of its own, does not appear to approach it within the ranges the publisher is capable of sustaining. This is consistent with the findings of Burger et al. [59]. In the multi-publisher UDP topology, a single SR-IOV subscriber receives from three concurrent publishers at 16 000 data-points each, yielding an aggregate of 48 000 datapoints per interval, without exhibiting signs of saturation, further corroborating the characterization of the subscriber as the less constrained component. On the TCP side, the PI deployment could not sustain the 20 000 datapoint configuration that Burger et al. report as achievable, a discrepancy plausibly attributable to the computational overhead introduced by the debug logging infrastructure, though a definitive assessment would require measurements conducted without instrumentation and falls outside the scope of this thesis. Collectively, these observations indicate that CPU-side constraints manifest predominantly on the publisher side.

Bandwidth evaluation

The interface bandwidth measurements corroborate the CPU utilization results of Figure 38, as throughput degradation on the Raspberry Pi at high data point counts directly reflects the processing bottleneck observed there.

TCP. As shown in Figure 43, subscriber-side receive bandwidth scales linearly with data point count for both deployments up to 4000 data points. Beyond this point the Raspberry Pi throughput declines, consistent with the CPU saturation observed in Figure 38, while SR-IOV continues scaling to approximately 626 kB/s at 8000 data points. Publisher-side transmit traffic, shown in Figure 42, reflects only subscription request overhead and remains near zero throughout. No packet loss is observed, as the received bandwidth across three publishers matches the expected transmitted volume

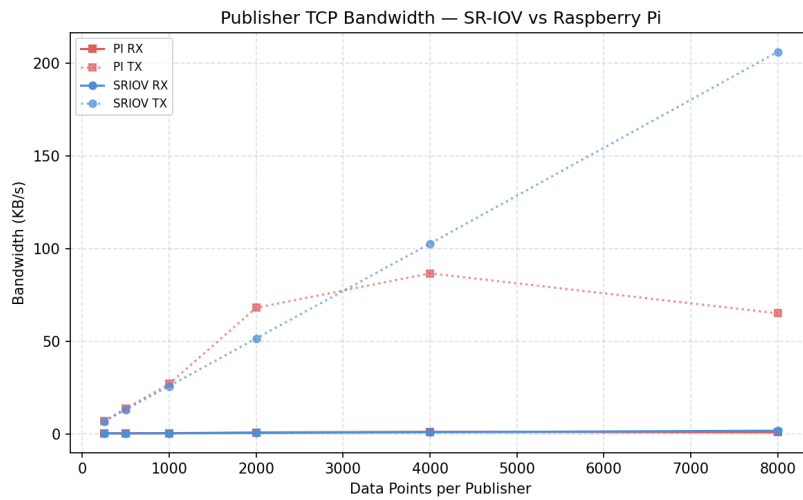


Figure 42: Average Publisher side bandwidth measurements for TCP. Tx for both PI and SR-IOV are not exactly zero as they are still sending publish requests. A drop off at the saturation point can be also observed. The elevated value at 2000 data points is attributed to measurement variance and is not considered representative of the underlying trend.

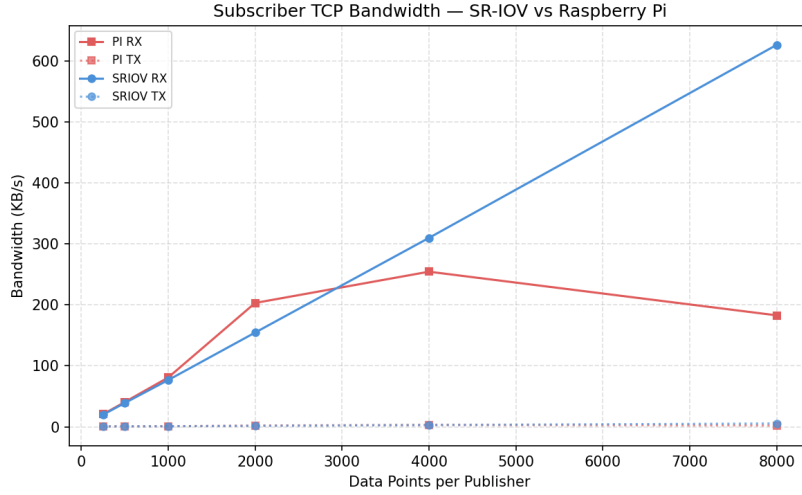


Figure 43: Subscriber side bandwidth measurements for TCP. The Tx and Rx values match $3\times$ the average Publisher values, indicating no packet loss in both systems.

UDP. Publisher-side transmit bandwidth, shown in Figure 44, scales linearly with data point count for both deployments, with no saturation effects observed across the measured range. Receive bandwidth on the SR-IOV publisher increases proportionally alongside transmit, reaching approximately 157 kB/s at 16000 data points, whereas Raspberry Pi receive bandwidth remains near zero throughout. This asymmetry reflects a difference in multicast handling between the two infrastructures as the physical switch forwards multicast frames exclusively to ports whose hosts have joined the corresponding multicast group, suppressing traffic at non-member ports. The SR-IOV testbed does not implement equivalent multicast filtering, causing all multicast frames to flood across the virtual switch and appear as receive traffic on the publishing VFs.

Subscriber-side measurements are shown in Figure 45. Receive bandwidth scales linearly with data point count for both deployments, and transmit bandwidth remains near zero throughout, consistent with the brokerless UDP transport producing no acknowledgement traffic. The correspondence between publisher transmit and subscriber receive confirms the absence of packet loss on both deployments.

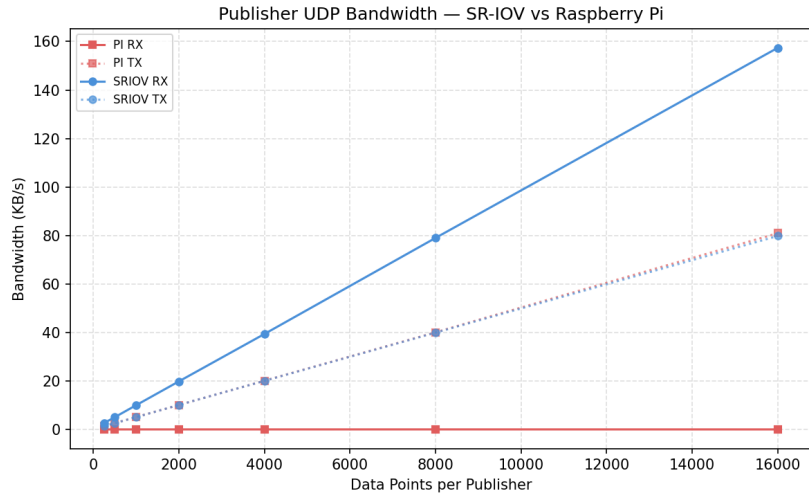


Figure 44: Publisher-side UDP bandwidth. SR-IOV receive traffic reflects multicast flooding in the absence of group-aware filtering.

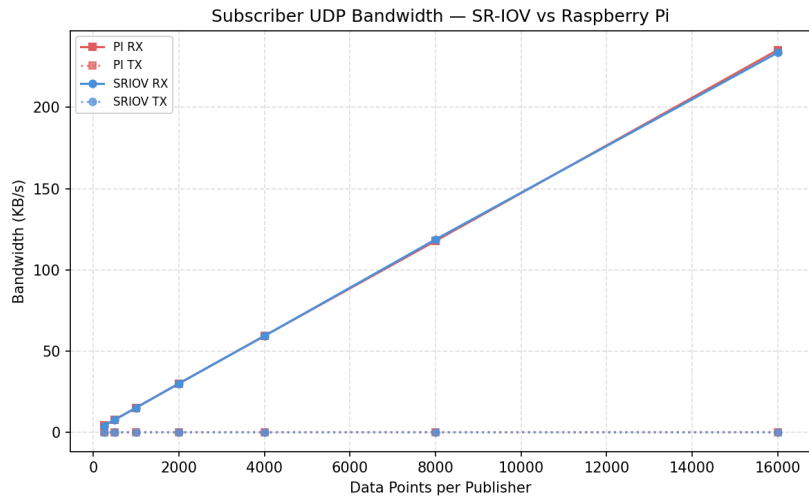


Figure 45: Subscriber-side UDP bandwidth. Tx remains at zero during the measurements.

Latency evaluation

Figures 46 and 47 present the median latency measurements for the TCP client-server and UDP publish-subscribe workloads, respectively, across increasing message payload sizes (quantified as datapoints per transmission). In both protocols, the SR-IOV deployment consistently yields lower observed

latency than the PI hardware baseline, with the two systems exhibiting qualitatively similar scaling behavior across the evaluated range.

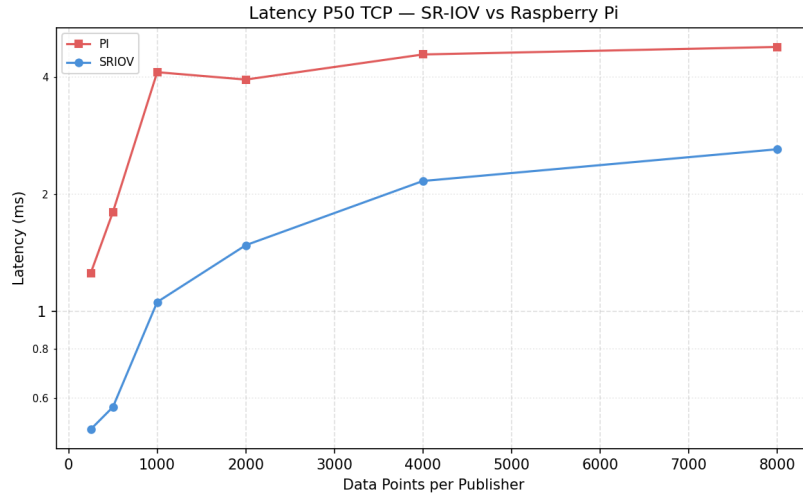


Figure 46: Median (p50) latency for the TCP configuration spanning multiple data points.

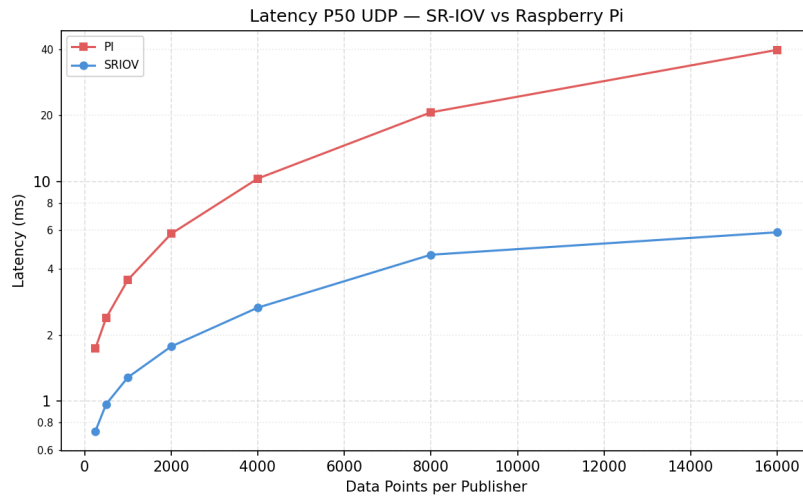


Figure 47: Median (p50) latency for the UDP configuration spanning multiple data points.

For the TCP workload (Figure 46), PI latencies rise from 1.255 ms at 250 datapoints to 4.780 ms at 8000 datapoints, while SR-IOV latencies increase from 0.498 ms to 2.610 ms over the same range. The ratio of PI to SR-IOV

median latency spans approximately $1.8\times$ to $3.9\times$, with a mean of $2.7\times$ across the six measurement points. Notably, both deployments exhibit a pronounced latency increase between 500 and 1 000 datapoints, followed by a sub-linear growth regime, suggesting that this inflection corresponds to a shared congestion or buffering threshold inherent to the protocol rather than a platform-specific artefact.

For the UDP workload (Figure 47), the divergence between the two deployments is more pronounced at larger payload sizes. PI latency grows near-linearly from 1.734 ms at 250 datapoints to 39.715 ms at 16 000 datapoints, implying an approximately linear relationship between message volume and queuing delay under this deployment. SR-IOV, by contrast, exhibits sub-linear growth over the same range (0.728 ms to 5.869 ms), which could be attributed to the aforementioned CPU saturation on the PI testbed. The PI-to-SR-IOV latency ratio consequently widens from $2.4\times$ at 250 datapoints to $6.8\times$ at 16 000 datapoints, indicating that absolute measurement parity should not be assumed at scale.

Nevertheless, a key observation pertinent to testbed validity is that both systems respond to increasing workload in a directionally consistent manner. Latency increases monotonically with datapoint count in both cases, and the relative ordering of the two deployments is preserved across all evaluated configurations. This monotonic co-variation implies that, within the TCP regime and the lower end of the UDP range, SR-IOV measurements can serve as a reliable ordinal proxy for PI performance, enabling trend identification and comparative protocol analysis without using dedicated hardware. It should be noted, however, that this represents a single case study conducted under controlled experimental conditions. The degree to which this correlation generalizes to alternative workload profiles, network topologies, or hardware configurations remains an open question, and broader validation across a wider parameter space would be necessary before drawing stronger conclusions.

6 Limitations and Future Work

Despite the contributions of this work, several constraints limit the generalizability and completeness of the results. These arise from a combination of hardware-specific restrictions, implementation decisions made due to time constraints, and deliberate simplifications of the experimental scope intended to keep the study tractable.

6.1 Scope of OPC UA Communication Models

This thesis evaluates only two of the communication mechanisms defined by the OPC UA specification: the brokerless UDP-based Publish-Subscribe transport using UADP encoding, and the session-based Client-Server model over TCP.

The broker-based PubSub transports, namely MQTT and AMQP, were not included in the evaluation as well as Client-Server HTTPS. Also only UADP encoding was used with the possible XML and JSON encoding being a possible alternative. As a result, the comparative analysis does not capture the performance trade-offs introduced by an intermediate message broker, which is a relevant deployment pattern in industrial environments. Future work should incorporate these broker-based transports, allowing a direct performance comparison between brokerless and broker-mediated communication under controlled conditions, particularly with respect to the latency and throughput penalties introduced by message routing and queuing at the broker.

6.2 Uncoordinated Publishing Intervals

The current implementation does not apply any coordination or staggering of publish intervals across multiple publishers. All publishers are configured with the same interval and begin transmitting after a shared startup delay, which causes their publish cycles to align and produce synchronized bursts of network traffic. This does not reflect an optimized deployment strategy, where publish intervals would typically be offset to distribute load evenly across time. The burst behavior artificially inflates peak bandwidth and latency measurements and may obscure the steady-state performance characteristics of the system. Future work should investigate coordinated interval scheduling to distribute load evenly across time, as well as adaptive strategies such as deadband-triggered or event-driven publishing, to better reflect realistic deployment behavior and expose the true steady-state performance characteristics of the system.

6.3 Single Subscriber

All experiments were conducted with a single subscriber or client collecting data from all publishers. While this simplifies the measurement setup and focuses the analysis on the network architecture, it does not reflect the one-to-many distribution scenarios that motivate the use of PubSub in the first place. The behavior of the system under multiple concurrent subscribers, particularly with respect to multicast replication and CPU load on the receiving side, remains uncharacterized. Future work should extend the application to support multiple subscribers and many-to-many topologies, which is a prerequisite for validating the testbed against the large-scale deployment scenarios that originally motivated its design.

6.4 Limited Physical Validation

The Raspberry Pi testbed used for physical comparison consisted of only four nodes, constraining the maximum publisher count in the comparative study to three. This limits the ability to validate the SR-IOV testbed’s fidelity at higher node counts, where resource contention and scheduling effects are more likely to diverge between the virtual and physical environments. Future work should extend the physical testbed to a larger number of nodes, allowing validation of the SR-IOV emulation accuracy across the full publisher range evaluated in the virtual setup.

6.5 Experimental Depth

The parameter study presented in this thesis is intended to demonstrate the measurement capabilities of the testbed rather than to provide statistically rigorous conclusions. Each configuration is measured in a single run, and no repeated measurements are taken to characterise variance or account for outliers. The results should therefore be interpreted as indicative rather than definitive. Furthermore, only a single subscriber is considered across all experiments, as multi-subscriber scenarios are out of scope for this thesis. The findings establish that the testbed is capable of producing clear and distinctive measurements across a range of configurations, but an in-depth statistical analysis is left for future work.

6.6 Quantifying Testbed Fidelity Against Physical Reference

Future work could extend the physical validation by deriving a scalar fidelity metric that quantifies the agreement between the SR-IOV testbed and the Raspberry Pi reference measurements, removing the dependence on visual inspection of distribution plots. Two candidate approaches are the two-sample Kolmogorov-Smirnov test applied to normalised latency distributions, and the Scalability Index introduced by Hoffeld et al. [72], which normalises the scalability behaviour of a system under evaluation against a reference system and produces a single dimensionless index over a defined range of the scaling parameter.

7 Conclusion

This thesis designed, implemented, and evaluated an SR-IOV-based virtual testbed constructed across two SR-IOV-capable Intel E810 NICs and suggests that such an architecture is capable of reproducing network-level trends observed on physical hardware. The comparison between the SR-IOV and Raspberry Pi deployments in Section 5.6 indicates that both systems respond to increasing data point counts in a directionally consistent manner across both TCP and UDP protocols. Latency increases monotonically with payload size in both environments, and the relative ordering between deployments is preserved across the evaluated configurations. Within the TCP regime and the lower end of the UDP range, the SR-IOV testbed therefore appears to function as a reliable ordinal proxy for physical hardware performance, suggesting that protocol trend identification is feasible without requiring dedicated embedded nodes for every experiment.

The testbed supports up to 119 concurrent publisher containers on a single physical host, with the theoretical ceiling imposed by the 128 virtual functions available per NIC port. Publisher scaling experiments suggest that individual per-publisher transmission rates remain stable across the full evaluated range, and subscriber-side bandwidth scales proportionally without observed packet loss. Latency was measured at the application layer by design, reflecting end-to-end delay as experienced by a consumer of the OPC UA stack. At high publisher counts, the debug-mode logging required to extract application-layer timestamps imposes overhead that compounds with publisher and writer group count, and for larger deployments a more lightweight instrumentation mechanism would be necessary to obtain unconfounded results. The demonstrated capacity to instantiate and coordinate over one hundred isolated networked application instances on a single host nonetheless appears promising for large-scale industrial protocol evaluation.

In the many-to-one configuration evaluated throughout this study, the TCP-based Client-Server model appears to yield lower median latency and more predictable tail behavior than the UDP-based Publish-Subscribe transport across the parameter studies conducted. TCP median latency in the data point scaling study increases from 0.36 ms to 1.50 ms as payload grows from 25 to 4000 data points, while UDP median latency fluctuates between 15 and 24 ms with no consistent trend, an elevation attributable primarily to logging queue delay rather than network transmission time. Under publisher scaling, UDP subscriber latency grows at approximately 100 ms per additional publisher beyond 10 nodes due to the polynomial scaling of per-packet debug log output, whereas TCP subscriber latency decreases from 0.59 ms to 0.26 ms, which may reflect more efficient notification batching un-

der higher server load. The results therefore suggest that TCP is sufficient, if not preferable, in many-to-one topologies within the evaluated parameter space. In many-to-many scenarios, where TCP session management overhead grows with each additional subscriber while UDP multicast traffic remains independent of receiver count, this ordering would be expected to invert, and validating that expectation within the testbed remains an important next step.

The topology comparison in Section 5.5 indicates that the testbed reproduces the expected structural ordering of latency across star, line, and tree topologies. The star topology yields the lowest observed median latency at $528\text{ }\mu\text{s}$, the tree $541\text{ }\mu\text{s}$, and the line the highest at $571\text{ }\mu\text{s}$, which is consistent with the path length differences imposed by each forwarding structure. Bandwidth appears unaffected by topology across all three configurations, suggesting that the software-defined forwarding layer on the switch node influences path latency without measurably affecting throughput. The absolute differences span only $43\text{ }\mu\text{s}$, a consequence of the forwarding steps being realized entirely in software on the same physical host without propagation delay or link emulation. The directional ordering is preserved and structurally consistent, but the compressed dynamic range limits the extent to which these results can be generalized to physically separated deployments. Incorporating a network emulation layer such as `netem` [73] to inject configurable per-hop delay and loss would produce more pronounced and distinguishable results, and is a natural extension for future work.

References

- [1] M. Schleipen, S.-S. Gilani, T. Bischoff, and J. Pfrommer, “Opc ua & industrie 4.0 - enabling technology with high diversity and variability,” in *Procedia CIRP, 49th CIRP Conference on Manufacturing Systems (CIRP-CMS 2016)*, vol. 57, Elsevier, 2016, pp. 315–320. DOI: 10.1016/j.procir.2016.11.055.
- [2] T. Lennvall, M. Gidlund, and J. Akerberg, “Challenges when bringing IoT into industrial automation,” in *Proceedings of the IEEE Africon 2017*, Cape Town, South Africa: IEEE, 2017, pp. 905–910. DOI: 10.1109/AFRCON.2017.8095602.
- [3] OPC Foundation, “OPC Unified Architecture Specification Part 14: PubSub,” OPC Foundation, Specification OPC 10000-14, 2018. [Online]. Available: <https://reference.opcfoundation.org/Core/Part14/v104/docs/>.

- [4] S. Gallenmüller, E. Hauser, and G. Carle, “Prototyping prototyping facilities: Developing and bootstrapping testbeds,” in *IFIP Networking Conference*, IEEE, 2022. DOI: 10.23919/IFIPNetworking55013.2022.9829817.
- [5] N. Handigol, B. Heller, V. Jeyakumar, B. Lantz, and N. McKeown, “Reproducible network experiments using container-based emulation,” in *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies*, ser. CoNEXT ’12, ACM, 2012, pp. 253–264. DOI: 10.1145/2413176.2413206.
- [6] P. Emmerich, D. Raumer, S. Gallenmüller, F. Wohlfart, and G. Carle, “Throughput and latency of virtual switching with open vswitch: A quantitative analysis,” *Journal of Network and Systems Management*, vol. 25, no. 3, pp. 589–622, 2017. DOI: 10.1007/s10922-017-9417-0.
- [7] F. Wiedner, M. Helm, S. Gallenmüller, and G. Carle, “Hvnet: Hardware-assisted virtual networking on a single physical host,” in *IEEE INFOCOM Workshops: Computer and Networking Experimental Research using Testbeds (CNERT)*, IEEE, 2022. DOI: 10.1109/INFOCOMWKSHPS54753.2022.9798351.
- [8] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, ser. HotNets ’10, ACM, 2010. DOI: 10.1145/1868447.1868466.
- [9] OPC Foundation, “OPC Unified Architecture Specification Part 1: Overview and Concepts,” OPC Foundation, Specification OPC 10000-1, 2017. [Online]. Available: <https://reference.opcfoundation.org/Core/Part1/v104/docs/>.
- [10] OPC Foundation, “OPC Unified Architecture Specification Part 3: Address Space Model,” OPC Foundation, Specification OPC 10000-3, 2017. [Online]. Available: <https://reference.opcfoundation.org/Core/Part3/v104/docs/>.
- [11] IEEE, “IEEE Standard for Ethernet,” IEEE, Tech. Rep. 802.3-2022, 2022. [Online]. Available: https://standards.ieee.org/ieee/802.3-2008_Cor_1/4562/.
- [12] IEEE, “IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks,” Tech. Rep. IEEE Std 802.1Q-2022, Dec. 2022. DOI: 10.1109/IEEESTD.2022.10004498.

- [13] OPC Foundation, “OPC Unified Architecture Specification Part 4: Services,” OPC Foundation, Specification OPC 10000-4, 2017. [Online]. Available: <https://reference.opcfoundation.org/Core/Part4/v104/docs/>.
- [14] Embien Technologies, *A comprehensive guide to the OPC-UA standard*, <https://www.embien.com/industrial-insights/a-comprehensive-guide-to-the-opc-ua-standard>, Accessed: April 2026.
- [15] OPC Foundation, “OPC Unified Architecture Specification Part 6: Mappings,” OPC Foundation, Specification OPC 10000-6, 2017. [Online]. Available: <https://reference.opcfoundation.org/Core/Part6/v104/docs/>.
- [16] “MQTT Version 5.0,” OASIS, Tech. Rep., 2019, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [17] “AMQP Version 1.0,” OASIS, Tech. Rep., 2012, <https://docs.oasis-open.org/amqp/core/v1.0/amqp-core-v1.0.html>.
- [18] S. Deering, “Host Extensions for IP Multicasting,” IETF, Tech. Rep. RFC 1112, 1989, <https://www.rfc-editor.org/rfc/rfc1112>.
- [19] A. O. Richter, “Mitigating and resolving performance isolation issues of pcie passthrough and sr-ioV in multi-core virtualization,” Ph.D. dissertation, Technische Universität München, 2017. [Online]. Available: <https://mediatum.ub.tum.de/1356714>.
- [20] I. Ahmad, J. Anderson, A. Holler, R. Kambo, and V. Makhija, “An analysis of disk performance in vmware esx server virtual machines,” in *2003 IEEE International Conference on Communications (Cat. No.03CH37441)*, 2003, pp. 65–76. DOI: 10.1109/WWC.2003.1249058.
- [21] P. Barham et al., “Xen and the art of virtualization,” in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, ser. SOSP '03, New York, NY, USA: Association for Computing Machinery, Oct. 2003, pp. 164–177, ISBN: 1581137575. DOI: 10.1145/1165389.945462.
- [22] D. Gupta, L. Cherkasova, R. Gardner, and A. Vahdat, “Enforcing performance isolation across virtual machines in xen,” in *Proceedings of the ACM/IFIP/USENIX 7th International Middleware Conference*, M. van Steen and M. Henning, Eds., ser. Lecture Notes in Computer Science, vol. 4290, Berlin, Heidelberg: Springer, 2006. DOI: 10.1007/11925071_18.

- [23] B.-A. Yassour, M. Ben-Yehuda, and O. Wasserman, “Direct device assignment for untrusted fully-virtualized virtual machines,” IBM Research Division, Haifa Research Laboratory, Tech. Rep. H-0263 (H0809-006), 2008. [Online]. Available: https://www.researchgate.net/publication/228750224_Direct_device_assignment_for_untrusted_fully-virtualized_virtual_machines.
- [24] H. Raj and K. Schwan, “High performance and scalable i/o virtualization via self-virtualized devices,” in *Proceedings of the 16th International Symposium on High Performance Distributed Computing (HPDC '07)*, 2007, pp. 179–188. DOI: 10.1145/1272366.1272390.
- [25] Intel Corporation, *Intel 64 and ia-32 architectures software developer’s manual*, Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C and 3D, Dec. 2016.
- [26] D. Abramson et al., “Intel virtualization technology for directed I/O,” *Intel Technology Journal*, vol. 10, no. 3, 2006. [Online]. Available: https://www.researchgate.net/publication/236221584_Intel_Virtualization_Technology_for_Directed_IO.
- [27] PCI-SIG, *Pci express base specification*, Revision 3.0, PCI-SIG, Nov. 2010.
- [28] PCI-SIG, *Pci local bus specification*, Revision 3.0, PCI-SIG, Feb. 2004.
- [29] PCI-SIG, *Pci-x addendum to the pci local bus specification*, Revision 2.0, PCI-SIG, Jul. 2002.
- [30] J. Shafer et al., “Concurrent direct network access for virtual machine monitors,” in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 2007, pp. 306–317. DOI: 10.1109/HPCA.2007.346208.
- [31] PCI-SIG, *Single root I/O virtualization and sharing specification, revision 1.1*, PCI-SIG, Jan. 2010.
- [32] PCI-SIG, *Multi-root I/O virtualization and sharing specification, revision 1.0*, PCI-SIG, May 2008.
- [33] *Devlink e-switch attribute*, Linux Kernel Documentation. Accessed: 2026-04-21, The Linux Kernel Organization. [Online]. Available: <https://docs.kernel.org/networking/devlink/devlink-eswitch-attr.html>.

- [34] Intel Ethernet Products Group (EPG), *Intel® Ethernet Controller E810 eSwitch switchdev Mode: Technology and Configuration Guide*, Revision 1.0, 645272-001, Intel Corporation, Aug. 2021. [Online]. Available: https://cdrdv2-public.intel.com/645272/645272_E810%20eSwitch%20switchdev%20Mode%20TechConfigGuide_Rev1.0.pdf.
- [35] *NIC SR-IOV APIs*, Linux Kernel Documentation v6.12. Accessed: 2026-04-21, The Linux Kernel Organization. [Online]. Available: <https://www.kernel.org/doc/html/v6.12/networking/sriov.html>.
- [36] B. Hubert, T. Graf, G. Maxwell, R. van Mook, M. van Oosterhout, and P. B. Schroeder, *tc(8) – show / manipulate traffic control settings*, Linux man-page, part of the iproute2 suite, iproute2, 2001. [Online]. Available: <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [37] B. Hubert et al., *Linux advanced routing & traffic control howto*, Accessed: 2026-04-21, 2002. [Online]. Available: <https://lartc.org/howto/>.
- [38] K. Rechert and P. McHardy, “Vordränger: Queueing disciplines – traffic control mit linux,” *Linux-Magazin*, no. 2, pp. 28–37, 2005, Accessed: 2026-04-21. [Online]. Available: <https://www.linux-magazin.de/ausgaben/2005/02/vordraengler/>.
- [39] *Chapter 35: Linux traffic control*, Red Hat Enterprise Linux 9, Accessed: 2026-04-21, Red Hat, Inc. [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/linux-traffic-control_configuring-and-managing-networking.
- [40] *NAPI*, Linux Kernel Documentation. Accessed: 2026-04-21, The Linux Kernel Organization. [Online]. Available: <https://docs.kernel.org/networking/napi.html>.
- [41] “IEEE standard for local and metropolitan area networks – virtual bridged local area networks – amendment: Frame check sequence extension,” Tech. Rep. 802.3ac-1998, 1998. [Online]. Available: https://standards.ieee.org/standard/802_3ac-1998.html.
- [42] S. Nimmagadda, “Linux namespaces and cgroups as os primitives for lightweight virtualization: Architecture, isolation mechanisms, and performance evaluation,” *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 9, pp. 811–822, Aug. 2018. DOI: 10.61841/turcomat.v9i2.15258.
- [43] Docker Inc., *Docker Documentation*, <https://docs.docker.com>, Accessed: 2026-04-05.

- [44] I.-M. Stan, D. Rosner, and Ș.-D. Ciocîrlan, “Enforce a global security policy for user access to clustered container systems via user namespace sharing,” in *19th RoEduNet Conference: Networking in Education and Research (RoEduNet)*, IEEE, 2020. DOI: 10.1109/ROEDUNET51892.2020.9324866.
- [45] A. Kuznetsov et al., *Ip-link(8) – linux man page*, iproute2, accessed 2025. [Online]. Available: <https://man7.org/linux/man-pages/man8/ip-link.8.html>.
- [46] T. Nagendra and R. Hemavathy, “Unlocking kubernetes networking efficiency: Exploring data processing units for offloading and enhancing container network interfaces,” in *2023 4th IEEE Global Conference for Advancement in Technology (GCAT)*, IEEE, 2023, pp. 1–7. DOI: 10.1109/GCAT59970.2023.10353542.
- [47] A. Ayub, M. Ishaq, and M. Munir, “Enhancement in multus cni for dpdk applications in the cloud native environment,” in *2023 26th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*, IEEE, 2023, pp. 66–68. DOI: 10.1109/ICIN56760.2023.10073475.
- [48] S. Qi, S. G. Kulkarni, and K. K. Ramakrishnan, “Understanding container network interface plugins: Design considerations and performance,” in *2020 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN)*, IEEE, 2020, pp. 1–6. DOI: 10.1109/LANMAN49260.2020.9153266.
- [49] D. Haja, M. Szalay, B. Sonkoly, G. Pongracz, and L. Toka, “Sharpening kubernetes for the edge,” in *Proceedings of the ACM SIGCOMM 2019 Conference Posters and Demos (SIGCOMM Posters and Demos ’19)*, New York, NY, USA: Association for Computing Machinery, 2019, pp. 136–137. DOI: 10.1145/3342280.3342335.
- [50] Y. Dong, X. Yang, X. Li, J. Li, K. Tian, and H. Guan, “High performance network virtualization with sr-iov,” in *HPCA - 16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture*, 2010, pp. 1–10. DOI: 10.1109/HPCA.2010.5416637.
- [51] J. Li, S. Xue, W. Zhang, R. Ma, Z. Qi, and H. Guan, “When i/o interrupt becomes system bottleneck: Efficiency and scalability enhancement for sr-iov network virtualization,” *IEEE Transactions on Cloud Computing*, vol. 7, no. 4, pp. 1183–1197, 2019. DOI: 10.1109/TCC.2017.2712686.

- [52] S. Bauer, D. Raumer, P. Emmerich, and G. Carle, “Intra-node resource isolation for sfc with sr-iov,” in *2018 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, IEEE, 2018. DOI: 10.1109/CloudNet.2018.8549547.
- [53] S. Cirici, M. Paolino, and D. Raho, “Svff: An automated framework for sr-iov virtual function management in fpga accelerated virtualized environments,” in *2023 International Conference on Computer, Information and Telecommunication Systems (CITS)*, IEEE, 2023. DOI: 10.1109/CITS58301.2023.10188786.
- [54] L.-M. Wang, A. Zelezniak, E. S. Daniels, T. Miskell, and L.-D. Chen, “Build an sr-iov hypervisor,” in *2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, 2019, pp. 539–544.
- [55] A. T. de Oliveira Filho, E. Freitas, P. R. X. do Carmo, E. Souto, J. Kellner, and D. F. H. Sadok, “Analysis of sr-iov in docker containers using rtt measurements,” *Computer Communications*, vol. 228, p. 107961, 2024. DOI: 10.1016/j.comcom.2024.107961.
- [56] *Network function representors*, Linux Kernel Documentation. Accessed: 2026-04-21, The Linux Kernel Organization. [Online]. Available: <https://docs.kernel.org/networking/representors.html>.
- [57] The Linux Kernel Community, *Linux base driver for the intel® ethernet controller 800 series*, Accessed: 2025. [Online]. Available: https://docs.kernel.org/networking/device_drivers/ethernet/intel/ice.html.
- [58] Intel Corporation, *ice Linux* Base Driver for the Intel® Ethernet 800 Series*, <https://github.com/intel/ethernet-linux-ice>, GitHub repository. README dated December 11, 2025. Accessed: 2026-04-05, 2025.
- [59] A. Burger, H. Koziolk, J. Rückert, M. Platenius-Mohr, and G. Stomberg, “Bottleneck identification and performance modeling of opc ua communication models,” New York, NY, USA: Association for Computing Machinery, 2019, ISBN: 9781450362399. DOI: 10.1145/3297663.3309670. [Online]. Available: <https://doi.org/10.1145/3297663.3309670>.
- [60] J. Pfrommer, A. Ebner, S. Ravikumar, and B. Karunakaran, “Open source OPC UA PubSub over TSN for realtime industrial communication,” in *Proceedings of the 23rd IEEE International Conference on Emerging Technologies and Factory Automation*, ser. ETFA ’18, IEEE, 2018. DOI: 10.1109/ETFA.2018.8502479.

- [61] F. Rezabek, M. Helm, N. Buchner, M. Smolarska, B. Jaeger, and G. Carle, “Assessment of OPC UA PubSub at scale using TSN,” Technical University of Munich, Tech. Rep., 2024. [Online]. Available: <https://www.net.in.tum.de/fileadmin/bibtex/publications/papers/2024-OPC-UA-PubSub-TSN-Modeling.pdf>.
- [62] J. Liu, W. Rao, M. B.-Y. Michael, and H. F. Wang, “High performance network virtualization with SR-IOV,” in *Proceedings of the 16th International Symposium on High-Performance Computer Architecture*, ser. HPCA ’10, IEEE, 2010. DOI: 10.1109/HPCA.2010.5416637.
- [63] *Axsol gmbh*. [Online]. Available: <https://www.axsol.eu/>.
- [64] Intel Corporation, *Ethernet Port Configuration Tool (EPCT)*, In: *Intel® Ethernet Adapters and Devices User Guide*, Accessed: 2026-04-05. [Online]. Available: <https://edc.intel.com/content/www/us/en/design/products/ethernet/adapters-and-devices-user-guide/ethernet-port-configuration-tool-epct/>.
- [65] *open62541: Open source opc ua implementation*, o6 Automation GmbH, 2026. [Online]. Available: <https://open62541.org/>.
- [66] The Git Development Community, *Git Reference Manual*, <https://git-scm.com/docs>, Accessed: 2026-04-05.
- [67] Raspberry Pi Ltd, *Raspberry pi 3 model b+ product brief*, Accessed: 2026-04-26, 2018. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>.
- [68] Unified Automation GmbH, *UaExpert – A full-featured OPC UA client*, <https://www.unified-automation.com/products/development-tools/uaexpert.html>, Version 2.0.0, accessed April 2026, 2025.
- [69] Free Software Foundation, *GNU Screen*, 2024. [Online]. Available: <https://www.gnu.org/software/screen/>.
- [70] Docker Inc., *Docker CLI source: stats_helpers.go*, Accessed: April 2026, 2025. [Online]. Available: https://github.com/docker/cli/blob/master/cli/command/container/stats_helpers.go.
- [71] S. Godard, *Sysstat – system performance monitoring tools for Linux*, <https://github.com/sysstat/sysstat>.
- [72] T. Hossfeld, P. E. Heegaard, and W. Kellerer, “Comparing the scalability of communication networks and systems,” *IEEE Access*, vol. 11, pp. 101 474–101 497, 2023. DOI: 10.1109/ACCESS.2023.3314201.

- [73] S. Hemminger, *tc-netem(8) – network emulator*, iproute2. [Online]. Available: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>.